FROM THE

PDP-11

TO THE

MSP430

THE INDUSTRIAL COMPUTER, REIMAGINED



Hardware / Assembly / C / Python
Timing / Sensors / Industrial Control

FINAL EDITORIAL TEXTBOOK EDITION

2026 • COURSEBOOK + LABORATORY COMPANION

TECHNICAL LABORATORY SERIES

FROM THE PDP-11 TO THE MSP430

The Industrial Computer, Reimagined

Hardware • Software • Industrial Automation • Measurement

Final editorial textbook edition · October 2026

An independent educational textbook and instructor laboratory companion.

Publication and technical notice

Final editorial textbook edition, October 2026. Prepared for independent study and laboratory instruction. No ISBN or external publisher is asserted. The PDP-11 and DEC operating systems are discussed historically; DEC, TI, Raspberry Pi, and their products are identified solely to describe the technology.

Examples labeled CANDIDATE are educational firmware and require target compilation and on-bench electrical verification before they are used. The included Python examples and host-side controller logic may be tested without the physical board, but host testing does not establish timing or electrical behavior on the MSP430.

This course uses 3.3 V logic, a safe LED output, and low-voltage signals. It does not qualify any design for industrial safety or mains-voltage actuation. Inspect the physical board, power source, and signal mappings prior to any connection.

How to use this textbook

Each chapter begins with a guiding question and essential vocabulary, develops the theory using physical or historical examples, offers a source listing where appropriate, and ends with a field laboratory. The lab has a prediction stage, a bounded procedure, an evidence gate, and a reproducible worksheet. Instructor keys are collected in the back of the book rather than interleaved with student work.

The laboratory sequence is deliberately staged. A successful 1 kHz frequency measurement does not validate 1-Wire microsecond timing. A correct CRC does not prove temperature accuracy. A working software state machine is not a certified industrial controller.

Learning pathways and prerequisites

The course is designed for an instructor-led introductory embedded-systems class, a self-directed electronics bench, or an advanced historical-computing seminar. The same evidence standard applies to all three pathways, but the order of presentation may change.

Architecture pathway

Read Chapters 1–4 first, then simulate the PDP-11 loop and interpret actual MSP430 assembly encodings before wiring a board.

Physical-computing pathway

Begin with the safely verified 1 kHz apparatus in Chapter 5. Return to the historical architecture chapters as the significance of addressing and interrupts becomes evident.

Instructor/operator pathway

Use Chapter 13, the laboratory evidence sheets, and Appendix F before the first class. Assign the physical experiment only after the operator readiness gate has passed.

Core hardware and software

• Target: MSP430G2553, custom lab board or independently verified compatible LaunchPad arrangement, and a known-good debugger.

• Instrument: Raspberry Pi Pico/Pico W with suitable frequency-counting firmware and, for 1-Wire, a separate verified pulse-capture capability.

• Host: development toolchain, a text editor, a compatible UART interface, and Python for analysis.

• Optional: PDP-11 emulation with correctly matched assembler and simulator documentation.

• Sensor: verified externally powered DS18B20 package, safe 3.3 V wiring, suitable pull-up, and independent thermometer.

Evidence maturity labels

• REFERENCE — documented historical or manufacturer specification.

• PAPER MODEL — computation or simulation, not an observation of physical hardware.

• CANDIDATE — code requiring compiler, wiring, and timing validation.

• HOST-TESTED — software logic tested on a computer, not on the target.

• BENCH-VERIFIED — an actual captured result with recorded setup and conditions.

Contents

Front matter • Preface • How to use the book • Chapter and laboratory sequence • Instructor keys • Appendices

Introduction — An engineering history that can be measured 2

Part I — The industrial computer and its architecture 4

Chapter 1 — Why the PDP-11 mattered in industrial automation 5

Chapter 2 — The PDP-11 programmer's view 13

Chapter 3 — The PDP-11 operating systems and the discipline of time 20

Chapter 4 — PDP-11 and MSP430: kinship and tradeoffs 27

Part II — The MSP430 becomes a physical experiment 34

Chapter 5 — Build a one-kilohertz signal, then measure it 35

Chapter 6 — Memory-mapped I/O: from an instruction to a wire 42

Chapter 7 — Real time is a promise, not a clock speed 49

Chapter 8 — A machine that can report what it does 56

Chapter 9 — One wire, two participants, and a microsecond-scale contract 63

Chapter 10 — Bytes become temperature: integrity, units, and meaning 71

Chapter 11 — Industrial control, hysteresis, and failure behavior 79

Chapter 12 — Publish an experiment that another engineer can reproduce 87

Part III — Teacher’s and operator’s companion 95

Chapter 13 — Twelve-session teaching plan 96

Chapter 14 — The historical bridge: what survives fifty years? 98

Appendices 99

Appendix A — Register comparison 99

Appendix B — Safe bench 99

Appendix C — CAP worksheet 99

Appendix D — References and provenance 100

Appendix E — Source inventory 100

Appendix F — Instructor answer key 101

Appendix G — Glossary and chapter locator 114

Appendix H — Engineering reference atlas 115

PREFACE / WHY THIS BOOK EXISTS

Preface — The instrument, the controller, and the historical question

In 1970, Digital Equipment Corporation introduced the PDP-11 to a world in which minicomputers were becoming practical instruments of science, manufacturing, and process control. Not every application demanded a mainframe. A smaller computer, equipped with a useful instruction set, memory, a modular I/O system, and software that responded to external events, could measure a physical process and participate in its control. That possibility reshaped laboratories and industrial engineering.

More than half a century later, a small MSP430G2553 microcontroller can reproduce many of the *conceptual* operations that made the PDP-11 attractive: read and write memory-mapped registers, respond to interrupts, perform arithmetic on sensor values, and act upon an event at a pin. The MSP430 is not a miniature PDP-11. The processors differ profoundly in bus architecture, resources, privilege models, peripheral arrangements, and operating environment. But they belong in a productive conversation about what a general-purpose instruction set can accomplish in the physical world.

This book places three machines in deliberate roles. The PDP-11 is the historical and architectural example, explored through documents and optional simulation. The MSP430G2553 is the physical target, with instructions, clocks, pins, and peripherals to understand. The RP2040/Pico is an observer and test instrument, not a substitute for the controlled device. The PC, using Python and familiar development tools, analyzes observations and produces engineering records.

The course adopts CAP: **Capture** raw observations with setup conditions; **Analyze** the calculations, limits, errors, and uncertainty; **Publish** enough information for somebody else to reproduce the work. The pedagogical sequence repeatedly begins with a prediction, then reveals a few relevant instructions, and only then moves to data.

The book is written for teachers, advanced learners, and practitioners who want to understand computers close to the hardware. It is also suitable for a self-directed bench course. A student does not need to know a PDP-11 before starting. A teacher does need the discipline to distinguish an example from an experiment and a measurement from an assertion.

Conventions and evidence labels

Historical example means a representative concept or original source, not a claim that the identical code ran on a particular industrial installation. **Candidate firmware** means plausible target code requiring a specific compiler, linker setup, board pin map, and bench verification. **Host-tested** means the included Python algorithm unit tests were executed without microcontroller hardware. **Verified on bench** may be claimed only when the teacher or learner has preserved the actual programming log, wiring record, instrument configuration, and measurements.

Code listings use their own syntax; do not paste a PDP-11 listing into an MSP430 assembler. The user's board pin assignments are treated as the proposed custom-board netlist, not universal MSP430 pin definitions. Unless stated otherwise, arithmetic exercises are examples, not actual readings.

Course outcomes

By the end, a learner should be able to explain a processor register, decode an addressing mode, distinguish an instruction from a peripheral operation, map control bits to a physical pin, compare C with generated machine code, reason about interrupt deadlines, verify a 1-Wire transaction electrically, decode and check a sensor value, and

publish a defensible control-system experiment. A student should also recognize that *historical similarity is not proof of direct architectural descent*.

Introduction — An engineering history that can be measured

The theme of this book is that good computing architecture is best understood by following a real signal. A student might admire the clean encoding of a PDP-11 instruction, but the deeper achievement was joining hardware, programming tools, and industrial peripherals into one serviceable system. The challenge of an industrial computer was never simply to calculate a numerical answer. It was to calculate the right result from valid inputs, within the time allowed by the physical process, and to communicate or act on that result through an interface that could fail.

The machine in its environment

A steel rolling mill illustrates the distinction. Sensors can watch temperature and line speed while identification data associates a reading with a particular piece of material. Actuating hardware can alter a setpoint, operate an alarm, or request intervention. A supervisory computer helps organize these activities, but it is not the sensor, the motor drive, the emergency stop, or the instrument by which its own behavior is tested. This separation was important during the minicomputer era and remains important when a microcontroller costs very little.

For this book, the historical computer is the PDP-11; the learning processor is the MSP430G2553; and the independent measuring device is a Raspberry Pi Pico. That is not a claim that three machines are interchangeable. It is a practical division of jobs. The PDP-11 provides a rich history of memory-mapped hardware and event-driven software. The MSP430 exposes comparable concepts through a small instruction set and on-chip timers. The Pico makes timing and edges observable. A host PC offers programming, notebooks, tests, and formatted publication.

Five questions for every experiment

What physical quantity or event matters? Start with the phenomenon: a pulse, a switch closure, or a changing temperature. **What does the computer represent?** Identify the bit pattern, register, memory address, and timing reference. **What action will the program request?** Draw the causality from instruction to peripheral to voltage. **What can the independent observer establish?** Distinguish a waveform's frequency from pulse-width accuracy, and a valid CRC from sensor accuracy. **How will the result be reproduced?** Save versions, pin maps, clock assumptions, raw observations, and a bounded conclusion.

These questions produce a book that can be used on a yellow legal pad when hardware is absent. Every lab permits a manual prediction and an order-of-magnitude estimate before the electrical result is available. Actual measurements remain irreplaceable for claims about actual hardware.

How the course is arranged

Part I establishes the PDP-11 industrial and architectural precedent. Part II follows a sequence of progressively richer MSP430 experiments: a timer signal; GPIO and switches; asynchronous event handling; UART telemetry; 1-Wire electrical signaling; sensor decoding; and control with hysteresis and fault policy. Part III gives instructors a reproducible sequence and evaluation framework. Appendices provide register references, safety constraints, test evidence, and code inventory. Students may enter through the physical experiment and return to the history after seeing the need for addressing modes and interrupts.

The final objective is not to build an industrial-certified controller. It is to produce a small, understandable demonstrator whose electrical behavior, source code, data interpretation, and limitations can be explained and tested. A correct laboratory result is an evidence-backed statement, not a marketing claim.

INDUSTRIAL COMPUTING / FOUNDATIONS AND PRACTICE

Part I — The industrial computer and its architecture

HISTORY • ARCHITECTURE • MEASUREMENT • CONTROL

CHAPTER 01 / CORE READING

Why the PDP-11 mattered in industrial automation

Guiding question: A working machine, not an isolated CPU

Key vocabulary: Industrial process, transducer, I/O interface, control path, evidence path

1.1 The industrial problem before the processor

Consider a tank whose contents must be held between two temperature limits. A human operator could periodically read a thermometer, note a change, and adjust a valve. But the process might demand decisions faster than an operator can make them, or it might require hundreds of simultaneous observations, each recorded and associated with a timestamp. Earlier solutions used electromechanical controllers, analog electronics, dedicated hardware, and specialized process computers. The broader industrial challenge was to make measurement, judgment, and action repeatable.

A programmable minicomputer changed the economics of this problem. One computing system could perform multiple algorithms in software, collect measurements, produce logs, supervise alarms, and be reconfigured without rewiring every decision. That did not abolish sensors, signal conditioning, relays, or safety interlocks. Rather, it made the control logic programmable within a larger engineered system. A computer earns its place in that system only when its failure modes and deadlines are understood.

DEC's PDP-11 family was a particularly influential example. It offered 16-bit computing, a useful and comparatively orthogonal instruction set, modular interface options, and a growing ecosystem of programmers, peripheral equipment, and operating systems. Later models differed from the original PDP-11/20; this was a *family*, not a single fixed machine. A unit installed for laboratory acquisition might have very different memory, interfaces, and operating software from one used in a manufacturing plant.

1.2 A canonical industrial control chain

Physical process → transducer → signal conditioning → interface → processor → output interface → actuator → physical process. Every arrow represents an engineering contract. A transducer converts temperature, pressure, force, or position to a measurable electrical quantity. The input interface gives the computer sampled data or event signals. Software evaluates the data using a policy. The output interface can command a valve or motor, subject to interlocks that should not rely solely on a software decision.

A separate observer may record time, raw samples, actuator state, fault flags, and energy consumption. This distinction between *control* and *observation* will matter when we put a Pico beside the MSP430. An instrument that reports 1,000 Hz has not demonstrated that the controlled process is safe; a controller that lights an LED has not established the fidelity of a sensor.

1.3 UNIBUS, interrupts, and direct memory access

PDP-11 systems used modular interconnects including UNIBUS and, in other family members, Q-bus. The original UNIBUS was more than a convenient connector. It organized requests for bus ownership, data movement, peripheral registers, and interrupts. A disk or instrumentation interface could gain bus access; an interrupt could signal that an external condition needed service; direct-memory transfers reduced repeated CPU involvement for eligible peripherals. DEC's *PDP-11 Bus Handbook* documents bus arbitration and prioritized service in technical detail.

One powerful architectural idea was **memory-mapped I/O**. A peripheral control register could be accessed with a load or store instruction rather than a completely separate I/O instruction set. A programmer who understood addressing and the meaning of each device register could operate a hardware interface. The processor's instructions became the vocabulary for changing external machinery. In today's MSP430, register names such as `P1DIR`, `TA0CCR0`, and `UCA0TXBUF` serve an analogous *programming role*, though there is no external UNIBUS and the actual peripherals are different.

Another idea was **priority**. Not every event could wait for a periodic software poll. A machine could interrupt ongoing work to service an event; a real-time operating system could schedule a task with greater urgency than routine reporting. This did not guarantee deadlines by itself. Priority rules, worst-case service time, shared data, and I/O latency still determined whether the control system met its obligations.

1.4 Industrial history without mythology

The PDP-11's industrial importance can be described precisely without calling it a universal solution. It appeared in laboratories, test stands, process monitoring, manufacturing, and machine control. DEC manuals for real-time systems explicitly discussed applications such as refinery and steel-processing control. But a successful installed system was never just a CPU and an operating-system disk. It included electrical isolation, transducers, operator interfaces, maintenance procedures, field wiring, software revisions, and emergency fallbacks.

This is the standard we should import into a teaching lab. We do not demonstrate industrial reliability by making a thermostat LED blink. We demonstrate a first principle: deterministic local decisions, explicit sensor validity, controlled output states, observed timing, and evidence of what happens on failure. Safety-critical industrial control requires far more engineering and certification than this book attempts.

1.5 Worked exercise: sketch the process

Draw the temperature-control chain on a sheet of paper. Label: (1) actual temperature, (2) DS18B20 physical transducer, (3) 1-Wire electrical bus, (4) MSP430 state machine, (5) logical cooling request, (6) optional low-voltage fan driver, (7) Pico measurement point, (8) PC observation and report. Ask what remains functional if the PC is unplugged. The desired answer: the MSP430's local state machine must continue, but the observer can lose logging. Then ask what occurs if sensor communication fails. The output must enter a defined, documented state, not silently continue trusting old data.

CAP prompt. Capture a labeled system drawing; analyze which connections carry data versus physical energy; publish the reason that an unresponsive operator dashboard must not become an undefined actuator command.

Instructor note. Describe the PDP-11's industrial accomplishment accurately while separating it from the much smaller educational demonstrator. Do not equate a DS18B20's internal digital conversion with the modular A/D equipment used in many PDP-11 installations.

Case study: an imagined rolling-mill inspection station

Suppose a continuously moving steel strip passes a thermal inspection station. A pyrometer produces a temperature signal, a length encoder tracks the strip position, and an alarm must be associated with the correct section of material. A computer can help associate a particular sample with a position, apply the inspection rule, and make a record. That does not imply the computer itself should directly energize heavy machinery. A physically independent protective circuit is a separate engineering obligation.

For the course, reduce the plant to a harmless model. A learner presses a switch to represent the arriving strip; a timer counts intervals; a temperature reading is captured; an LED signifies that a limit has been exceeded. The value of the exercise is preserving the full information chain, including time and uncertainty, while stripping away hazardous actuation.

The important historical contribution of the PDP-11 was not a particular factory algorithm. It was an adaptable and maintainable combination of CPU, I/O, storage, software, and application engineering. The computer could be reprogrammed to solve a different instrumentation problem without redesigning the entire decision apparatus.

Industrial automation: why an entire computer system mattered

A PDP-11 installation could serve as a data-collection station, a supervisory controller, a laboratory processor, or the host of a larger process-control system. These roles must not be conflated. In a data-acquisition station, measurements may be recorded for later decisions. A supervisory controller can propose setpoints for local controllers. In direct digital control, the computer participates more directly in choosing actuating commands. Each arrangement requires a different response to a lost sample, a broken cable, or a processor restart.

The DEC architecture made these configurations practical because the machine was not limited to one fixed peripheral arrangement. UNIBUS connected multiple devices in a consistent architectural framework; bus-mastering interfaces could move data efficiently; vectored interrupts allowed peripheral events to direct software attention. This is system architecture, not merely a fast instruction set. A laboratory temperature task does not require a UNIBUS, but it benefits from the same habit of defining a processor-to-device contract.

Use the historical rolling-mill case as an engineering design exercise. First separate measurements (thermocouple or pyrometer), identification (the strip section being measured), calculation (a limit and rate-of-change rule), and action (alarm or process-control request). Then add an independent protective layer for dangerous outcomes. Finally record which statements are historical capabilities and which are features of the classroom model. This discipline prevents an attractive analog from becoming an invented historical reconstruction.

PDP-11 hardware and the industrial interface contract

A CPU instruction that reads a device register is only the last link of a long measurement chain. Analog inputs require excitation or signal conditioning, conversion, appropriate reference voltages, grounding, and shielding. Digital contacts require defined polarity, isolation where required, and a known response to an open circuit. An external device may be slower than a CPU; software must account for settling, busy status, completion flags, and lost interrupts.

A useful classroom specification identifies, for each input, its physical variable, electrical representation, engineering units, normal range, valid-update interval, failure state, and timestamp origin. For each output it records the requested action, actuator interface, safe state, and independent limits. The PDP-11 hardware tradition was valuable because programmable logic and industrial interfaces could be integrated, not because software could ignore the plant. The MSP430 student should adopt exactly this interface-contract habit at a far smaller scale.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

1.1 Identify the input, calculation, output, and safety system in a hypothetical refinery tank controller; which parts should remain operational if the console is disconnected?

1.2 Explain the difference between *process temperature* and a digital temperature value; name three possible sources of discrepancy.

1.3 Create an interface contract for a switch that closes at most 15 times per minute. Specify active level, voltage, allowable delay, and response to broken wiring.

1.4 A sensor reports every 0.5 s. How many expected samples accumulate during a ten-minute shift? Which missing-sample patterns would a simple average conceal?

1.5 Describe why a PDP-11 UNIBUS device register and an MSP430 port register are similar programming abstractions but not interchangeable electrical components.

1.6 Design a small safe physical analog of a steel-mill inspection station using only a button, LED, and timer. State two limits of the model.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

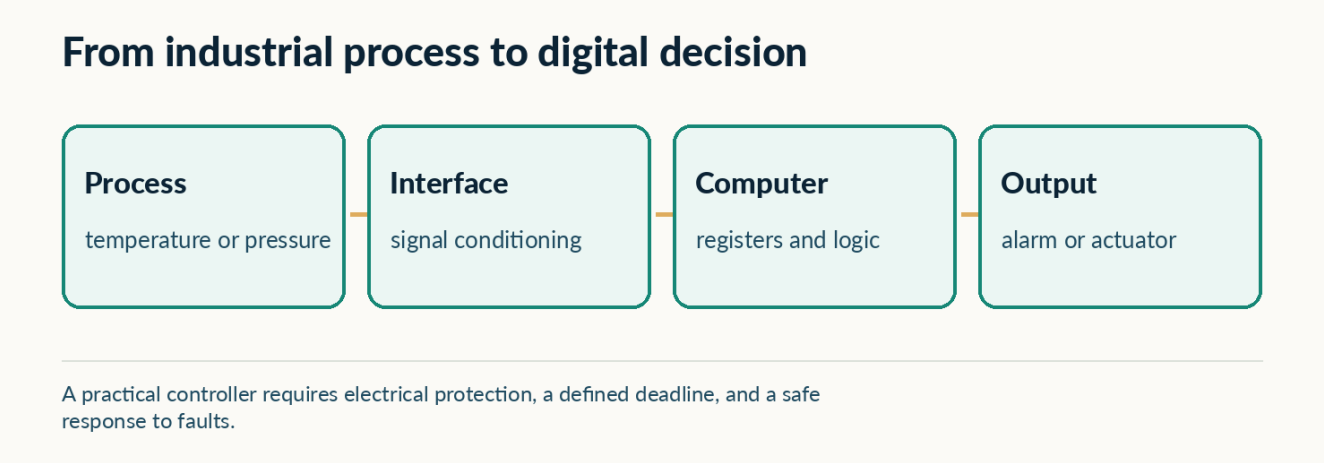


Figure 1.1 — Conceptual illustration for Chapter 1; not a verified wiring diagram.

Field laboratory 01: A Computer at the Steel Mill

Industrial-control anatomy and the PDP-11

Industrial or historical context

A rolling mill is a useful thought experiment because it combines measurements, delayed material transport, and outputs that have physical consequences. An imaginary sensor produces a pulse each time a piece reaches an inspection station. A second sensor reports a temperature. The computer must distinguish a product defect from an instrumentation defect. Historical PDP-11 installations existed in industries of this kind; the scenario in this lab is intentionally fictional, not a reconstruction of any installation. Beginning with a plausible process avoids the false implication that a minicomputer alone provides safe control. Experienced engineers will ask first about mechanical guards, interlocks, independent emergency stops, and who is allowed to energize an actuator.

Model and technical reasoning

Draw two paths: the CONTROL path from sensor to decision to actuator, and the EVIDENCE path from raw signal to timestamp to stored record. Place signal conditioning and galvanic isolation where needed. Then write a data dictionary with units, range, update rate, and the result expected if a sensor wire breaks. A temperature sample is not a decision; a limit is a policy; an output is an electrical command rather than a guarantee of mechanical motion.

Prediction before measurement

Predict which of these events requires the shorter response: a one-second temperature trend update, a mechanical limit switch, and a report requested from an operator console. Give an explicit deadline for the hypothetical system and distinguish your engineering choice from a historical DEC specification.

Procedure and evidence gates

1. Draw a process diagram showing physical plant, transducers, interfaces, CPU, outputs, and an independent observer.
2. Mark where conversion from engineering units to machine representations occurs.
3. Specify three faults: stuck-high input, stuck-low input, and a disconnected actuator.
4. Write an alarm and a separate fail-safe requirement that do not depend on the display workstation.
5. Label which pieces of this system could be emulated on the MSP430 laboratory bench and which require an industrial design beyond the course.

Interpretation questions

1. What would an oscilloscope prove that a printed telemetry line cannot?
2. Which components continue operating if the host PC powers down?
3. Why is a watched software loop not the same as an emergency-stop circuit?
4. What historical advantage did modular DEC interfaces offer plant engineers?

Required submission: One annotated block diagram; one interface contract table; a priority/deadline table; a fault-response sketch; a signed statement of what was assumed rather than observed.

WORKSHEET 01 / REPRODUCIBLE LAB RECORD

A Computer at the Steel Mill

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 01 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

A Computer at the Steel Mill • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: What would an oscilloscope prove that a printed telemetry line cannot?

_____

_____

Prompt: Which components continue operating if the host PC powers down?

_____

_____

Prompt: Why is a watched software loop not the same as an emergency-stop circuit?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 01 / DESIGN AND REASONING

The missing sensor exercise

In the simulated inspection cell, one of two signals is silently lost. Define a watchdog that detects absence of an expected pulse, while explicitly stating what it cannot infer about the real plant. Model three cases: normal arrivals, absent arrivals because the process stopped, and an unplugged sensor. A missing event alone cannot distinguish these cases. Add an independent heartbeat or second signal to improve the diagnostic.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Submit a three-state detection table, a timestamped artificial event sequence, a description of the ambiguity, and a justified operator alert. Do not call the result an emergency interlock.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 02 / CORE READING

The PDP-11 programmer's view

Guiding question: A program is a controlled change of machine state

Key vocabulary: Octal, registers, program counter, stack pointer, SOB, status flags

2.1 Eight registers and a coherent model

The classic PDP-11 programmer sees eight 16-bit registers. R0–R5 are available for general computations; R6 is the stack pointer (SP); R7 is the program counter (PC). The processor status word records condition information such as negative, zero, overflow, and carry. The fact that the PC and SP appear as addressable registers is not an incidental flourish. Combined with the addressing modes, it gives compact ways to express constants, relative addresses, stacks, and subroutine behavior.

Think of a register as an immediately accessible storage location inside the processor, not a variable in RAM. A `MOV` moves a value; `ADD` combines a source and a destination; condition codes describe the outcome. An instruction can operate on an ordinary register or a memory value selected through an addressing mode. The memory arrangement and peripheral mapping may vary by PDP-11 model and memory-management configuration; these introductory exercises avoid assuming an entire system layout.

2.2 Addressing modes are the real lesson

PDP-11 addressing modes include register, register deferred, autoincrement, autoincrement deferred, autodecrement, autodecrement deferred, indexed, and indexed deferred. On many double-operand instructions, the source and destination can use the same broad palette. That symmetry is called *orthogonality*: many operations can apply to operands selected in the same ways. Orthogonality is not absolute, and device or instruction constraints remain, but the design is remarkably elegant.

A register operand such as `R2` means the value currently held in the register. `(R2)` denotes the contents of memory at the address in R2. `(R2)+` accesses memory and then advances R2, typically by the operand size (with special cases). `4(R2)` adds an offset to the register to locate an operand. With PC-relative forms, the same principles can address code or data near the instruction itself.

The PDP-11's user-visible design demonstrates how architecture can reduce a programmer's need to write separate sequences for every kind of memory access. But supporting many addressing modes affects instruction encoding, hardware, timing, and memory transactions. We will see an MSP430 with more registers but fewer flexible destination modes.

2.3 Worked program: sum five integers

The short MACRO-11-style illustration in `code/pdp11/01_register_loop.mac` computes 5 + 4 + 3 + 2 + 1 = 15 decimal. R0 counts down; R1 accumulates. The `SOB` instruction subtracts one from a register and branches back if nonzero. Because the PDP-11 traditionally displays addresses and values in octal, 15 decimal is 17 octal. The listing is a teaching example, not a verified simulator session.

Read the program twice. On the first pass, write every value R0 and R1 will hold. On the second pass, count how many times `ADD` executes and explain why the branch eventually stops. Then identify the machine state at `HALT`. Notice that no variable called `sum` is necessary for this exercise: the registers themselves are the state.

2.4 Condition codes and signed arithmetic

A processor can represent a 16-bit pattern as an unsigned quantity or a signed two's-complement quantity. The bit pattern alone does not tell the programmer which interpretation is intended. Zero indicates a zero result; negative generally tracks the most significant bit; overflow matters when signed arithmetic cannot be represented; carry matters for unsigned arithmetic or multiword operations. It is a mistake to treat these status flags as interchangeable.

Exercise. Add 1 to 0x7FFF in a 16-bit signed context. What bit pattern results? Why does the overflow flag matter even if the resulting bit pattern can be read as a positive unsigned integer? Then subtract one from zero and discuss borrow and negative flag behavior using the processor manual's exact definitions.

2.5 Simulation laboratory

An optional Open SIMH PDP-11 configuration permits exploration without a physical minicomputer. Install a reputable emulator, follow its documentation to select a specific simulated CPU and console, and obtain a suitable assembler/OS environment or use documented front-panel deposit/examine functionality. **Do not assume that a MACRO-11 source file can be pasted directly at a SIMH prompt:** the simulator's command language is distinct from an assembler, and assembling software may require an operating-system image or another tool.

The simplest learning artifact is a register trace. Write the instruction, the before-state, the expected after-state, and the actual after-state. Avoid spending a full introductory week installing a historical operating system. An instructor can preconfigure the machine, or substitute a pencil-and-paper execution trace when infrastructure is unavailable.

CAP gate. Publish a register-by-register execution table and state whether the values came from a simulator, a documented hand trace, or an actual machine.

Worked trace: five additions and a branch

A trace is more instructive than an unexplained correct total. Begin with R0 holding 5 and R1 holding 0. After the first ADD, R1 becomes 5. SOB then decrements R0 to 4 and branches to the loop. The sequence of accumulator values is 5, 9, 12, 14, 15. The sequence of loop-counter values after SOB is 4, 3, 2, 1, 0. On the final SOB, the zero count suppresses the branch.

Why is the instruction sequence teachable? The loop counter resides in a register, arithmetic writes another register, and the PC changes according to a branch condition. Nothing here is mystical. The useful skill is accounting for each state transition. Trace the MSP430 counterpart with DEC R0 renamed to MSP430 R4; do not assume the PC, SP, or assembler syntax are interchangeable.

An emulator is especially useful when a real PDP-11 is unavailable. It lets the learner inspect registers at single-instruction resolution. However, a software emulation of register transitions does not establish the electrical latency of a physical PDP-11 peripheral interface.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

2.1 Trace the accumulator values for a PDP-11 register loop that adds 4, 3, 2, and 1. Record counter and program-counter decisions after each branch.

2.2 Convert decimal 63 to octal and hexadecimal, then write its 16-bit binary representation.

2.3 State the distinct roles of R6 and R7 on the PDP-11. What would a careless programmer risk by using them as scratch registers?

2.4 Explain how (R5)+ differs from R5 and from 4(R5); illustrate all three with a two-word array.

2.5 Compare signed and unsigned interpretation of the 16-bit pattern 0xFFFF. What additional status flags matter when interpreting an arithmetic result?

2.6 Write a short paper trace that distinguishes a simulated result from a measurement made on actual DEC hardware.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · pdp11/01_register_loop.mac

```
  1   ; MACRO-11-style listing for historical/ISA study. Not assembled or run.
  2   ; Arithmetic: 5+4+3+2+1 = 15 decimal (=17 octal).
  3           .TITLE  REGLOOP
  4           .PSECT  CODE
  5   START:  MOV     #5,R0
  6           CLR     R1
  7   LOOP:   ADD     R0,R1
  8           SOB     R0,LOOP
  9           HALT
 10           .END    START
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 02: A PDP-11 from the Programmer’s Chair

Registers, stack, octal notation, and an emulator

Industrial or historical context

The PDP-11 had eight visible 16-bit registers. R6 acted as stack pointer, R7 as program counter, and R0 through R5 were ordinary working registers. That compact and regular model remains one of the most teachable examples of assembly language. The PDP-11 community's use of octal can initially distract learners whose later work is mostly hexadecimal. Explain octal as a historical convention aligned to groups of three bits, not as a magical hardware feature. A value of decimal 15 is octal 17, and the same 16-bit pattern can be written in several bases.

Model and technical reasoning

Trace the loop in code/pdp11/01_register_loop.mac before running it. Initially R0=5 and R1=0. The ADD instruction adds the current count to the accumulator; SOB decrements the count and branches when the new result is nonzero. Record each iteration and the instruction that changes R0. The expected final arithmetic total is decimal 15. A halted simulator is evidence of instruction execution only when the program has actually been assembled, loaded, and inspected.

Prediction before measurement

Write the values of R0 and R1 after every execution of SOB, before opening any simulator. State whether the last iteration adds 1 or 0 and justify the order of operations.

Procedure and evidence gates

1. Inspect the MACRO-11-style source file and underline register operands and immediate values.

2. Use a PDP-11 simulator with documented CPU and loader configuration; select an appropriate assembler or a verified binary input.

3. Load the program and stop it at entry. Record PC, SP, R0 and R1.

4. Step ADD and SOB separately for the first two iterations; note PC changes when the branch is taken.

5. After HALT, inspect accumulator state, compare with paper arithmetic, and record the exact emulator command sequence.

Interpretation questions

1. How do the register aliases SP and PC affect ordinary arithmetic instructions?

2. Why might a programmable machine provide a single decrement-and-branch instruction?

3. What is gained by using an emulator first, and what does the emulator not prove?

4. What changes in the trace if R0 is initialized to 1?

Required submission: Source listing, a five-row trace table, final register dump, simulator command record, and an explanation of octal versus decimal representation.

WORKSHEET 02 / REPRODUCIBLE LAB RECORD

A PDP-11 from the Programmer's Chair

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 02 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

A PDP-11 from the Programmer's Chair • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: How do the register aliases SP and PC affect ordinary arithmetic instructions?

_____

_____

Prompt: Why might a programmable machine provide a single decrement-and-branch instruction?

_____

_____

Prompt: What is gained by using an emulator first, and what does the emulator not prove?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 02 / DESIGN AND REASONING

Octal, hexadecimal, and machine words

Select ten 16-bit patterns that include zero, 1, 7, 8, 255, 32767, 32768, and 65535. Display each in octal, hex, binary, and signed decimal. Trace how a conditional branch changes if a value is interpreted as signed rather than unsigned. Do not assume the programmer-visible representation is stored as ASCII characters.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Deliver a representation table and a two-paragraph explanation of what actually changes when the same bits are viewed in another base.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 03 / CORE READING

The PDP-11 operating systems and the discipline of time

Guiding question: The machine must answer while the plant keeps moving

Key vocabulary: RT-11, RSX-11M, interrupt, response time, deadline, worst case

3.1 A process does not wait for a terminal

A commercial accounting program can often wait for an operator to type a command. A machine monitoring molten steel or a pressure vessel has different obligations. A measurement may need to occur within a deadline; an alarm may need to preempt a report; a sensor may fail at the worst possible instant. The distinction is not merely whether a computer is *fast*, but whether its behavior is appropriately bounded and supervised.

The PDP-11 hosted several software environments. RT-11 served compact single-user, real-time and laboratory uses. RSX-11M and RSX-11S are particularly instructive for industrial history because they exposed ideas of tasks, scheduling, event-driven activity, and dedicated real-time environments. Other members of the family supported interactive computing, business applications, and UNIX development. We should not teach those environments as interchangeable or suggest every PDP-11 installation ran the same software.

3.2 Three kinds of time

Students often confuse clock frequency, execution time, and response latency. **Clock frequency** gives the rate of timing events driving some processor or peripheral. **Instruction execution time** depends on the actual instruction, selected operands, memory accesses, and implementation. **Response latency** is the time from an external event to a meaningful system reaction. A fourth term, **jitter**, describes variations in event timing or latency. Industrial systems need good answers for all four, not just an impressive processor clock specification.

Consider a system that samples pressure every 10 ms. A large average CPU capacity does not prove that every sample arrives on time. A priority inversion, slow interrupt routine, or unbounded busy wait may cause a deadline miss. A real-time operating system can help organize work but cannot remove physical constraints. A small bare-metal MSP430 can also meet selected deadlines if its program is simple, measured, and carefully controlled.

3.3 Interrupts as promises with costs

An interrupt lets the CPU abandon its current instruction stream at a defined point, save enough state, and execute service code. A reliable interrupt service routine (ISR) does not casually execute arbitrary work. It should usually do the minimum needed to capture a state or acknowledge a device, then defer expensive formatting, calculations, or communication until appropriate. The exact costs differ between PDP-11 models, MSP430 family members, clock settings, and software conventions.

In RSX-11M, task priorities and event mechanisms help coordinate activities. In an MSP430 lab, a hardware timer and ISR can provide a simpler demonstration of responsiveness. The lesson is not to recreate RSX-11M on a chip with 512 bytes of SRAM. It is to identify which responsibilities should happen promptly and which can wait.

3.4 Exercise: a scheduling timeline

Draw a 100-ms timeline with a periodic sampling event at 0, 10, 20, ... 90 ms; add a serial logging task that takes 15 ms of CPU time and a higher-priority alarm needing immediate attention. Use a colored pencil to show what happens

if the program simply delays 15 ms at an inconvenient moment. Redraw with an interrupt that records the urgent event and with a foreground loop that handles logging. Then list assumptions: interrupt masking, time spent in ISR, interrupt priority, and whether logging can be interrupted.

This is a model, not a measured MSP430 timing result. The physical experiment in Chapter 7 will test a real program and show where our model is incomplete.

CAP gate. Publish a timeline annotated with deadlines, expected latency, and at least one plausible failure mode.

A scheduling example: observation, control, and reporting

Consider three hypothetical activities: a protection-related limit input, a 10 ms measurement update, and a one-second operator report. Rank their response requirements before proposing CPU utilization. Average instruction throughput alone does not answer whether a limit event can be serviced on time. The latency distribution, critical sections, interrupt masking, and competing peripheral operations matter.

The PDP-11 era made this distinction visible because real-time operating systems had to coordinate devices with deadlines while still supporting conventional programs. Today the MSP430 can demonstrate the same basic separation without hosting RSX-11M: a timer interrupt provides an event, the foreground loop performs low-urgency work, and an external observer measures the resulting timing.

Do not turn this into a blanket assertion that interrupts are always better than polling. Polling may provide simple, bounded behavior in a small controller. The choice depends on event rate, acceptable latency, interrupt interference, verification effort, and the work that must be done at each event.

RSX-11M as a real-time software case study

RSX-11M provided an operating environment for multiple tasks whose execution was related to events, priorities, and device activity. A useful mental model separates a rapidly serviced input from a periodic measurement computation and a relatively slow operator display. The highest-priority activity does not necessarily perform the most work; it handles the shortest deadline and passes compact information to other tasks.

Build a paper scheduling trace with three fictional jobs: an input alarm requiring a response within 2 ms, a measurement calculation released every 10 ms, and a status display released every 1 s. Add a 1 ms interval in which interrupts are disabled. The class must decide what can be concluded about worst-case alarm latency and what remains unknown about interrupt recognition, queueing, CPU service time, and actual hardware. There is no single correct schedule without these assumptions.

On an MSP430, the analogy can be explored with an interrupt service routine that copies an event flag or timestamp and returns, while foreground code handles reporting. This is not an implementation of RSX-11M and not equivalent to its task management. The historical point is that responsive industrial systems separate priority, latency, throughput, and noncritical reporting; the modern experiment puts measurable evidence under those abstractions.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

3.1 Assign response deadlines of your own to an emergency input, 100 Hz acquisition loop, and human-readable status report. Justify the deadlines in physical terms.

3.2 A timer service takes 150 microseconds every 10 milliseconds. What proportion of execution time does it consume if uninterrupted? Why is utilization not sufficient to prove deadlines?

3.3 Explain the difference among latency, jitter, execution duration, and frequency.

3.4 Draw a timeline in which a low-priority report is interrupted by a higher-priority input. Mark preemption, resume, and missed deadline if any.

3.5 Compare RT-11 and RSX-11M as historical programming environments without suggesting that a bare-metal MSP430 runs either operating system.

3.6 Propose one test that measures the distribution of response times instead of reporting only their mean.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

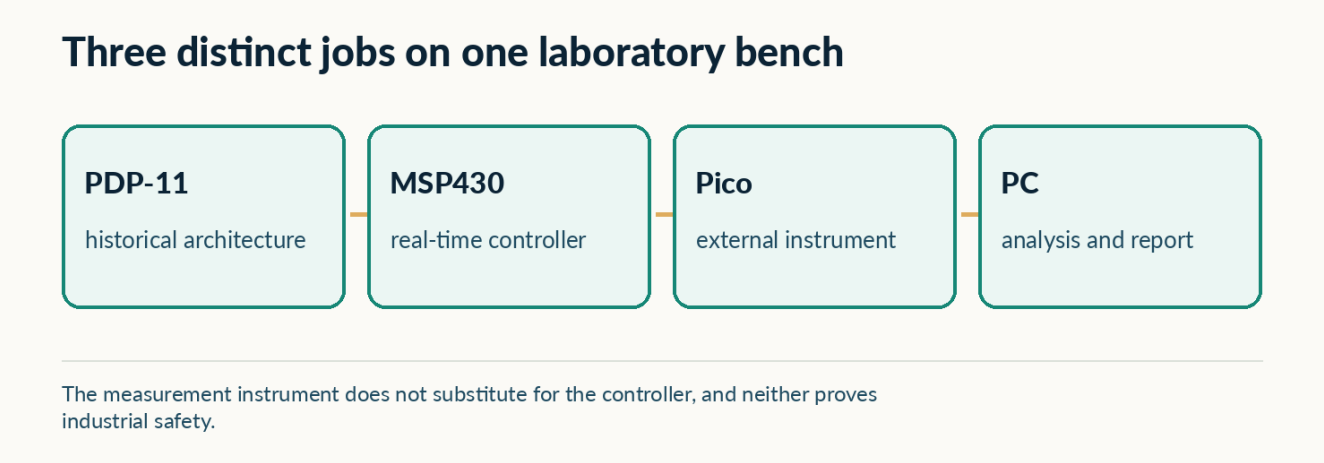


Figure 3.1 — Conceptual illustration for Chapter 3; not a verified wiring diagram.

Field laboratory 03: Two Instruction Sets in Conversation

PDP-11/MSP430 addressing-mode comparison

Industrial or historical context

The comparison becomes strongest when it moves beyond mnemonic resemblance. The PDP-11 uses three-bit register fields and three-bit mode fields to specify source and destination for many two-operand instructions. The MSP430 allocates four bits per register to expose sixteen architectural registers; its source receives more addressing-mode flexibility than its destination. The compact fixed word requires architectural compromises. The MSP430 R2 and R3 encodings recover useful constants and special forms without requiring a new memory word for every such value. That is a useful design tradeoff to examine without claiming that either chip was derived directly from the other.

Model and technical reasoning

Use a three-column translation table: programmer intent, PDP-11 expression, MSP430 expression. For example, add a value held in one register to another, access an indexed memory element, and walk a word array with post-increment. Then assemble at least two MSP430 examples and record each full instruction word and any extension words. Compare the binary cost of MOV.W #1,R5 with ADD.W #1,R5 in the chosen assembler. Do not infer actual encoding from the printed mnemonic alone.

Prediction before measurement

Predict which instruction forms could need extension words, and why a commonly used immediate constant could sometimes avoid them.

Procedure and evidence gates

1. Draw the PDP-11's eight registers and the MSP430's sixteen; label PC, SP, SR, and constant-generator roles.
2. Annotate MOV, ADD, and an indexed memory read in both syntaxes.
3. Take one instruction and manually mark opcode, source-register, addressing-mode, and destination fields according to the TI guide.
4. Build the MSP430 fragment under the installed assembler and save the map or listing.
5. Compare each prediction with the machine words and annotate all discrepancies.

Interpretation questions

1. Why is autoincrement in a source operand attractive for reading arrays?
2. Why is a destination-mode restriction a reasonable cost of having more registers?
3. What does an assembler pseudo-instruction conceal?
4. What evidence would be needed to claim a direct historical design influence?

Required submission: Paired code listings, encoding sketches, a hardware-specific assembler build log, and an explicit description of architectural differences.

WORKSHEET 03 / REPRODUCIBLE LAB RECORD

Two Instruction Sets in Conversation

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 03 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

Two Instruction Sets in Conversation • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: Why is autoincrement in a source operand attractive for reading arrays?

_____

_____

Prompt: Why is a destination-mode restriction a reasonable cost of having more registers?

_____

_____

Prompt: What does an assembler pseudo-instruction conceal?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 03 / DESIGN AND REASONING

Two deadlines and one processor

Create a scheduling simulation with an event every 5 ms requiring 600 µs of service and a reporting task every 100 ms requiring 8 ms of work. Assign priority explicitly and determine whether the proposed service arrangement could miss a deadline when an event arrives at the worst possible instant. Explore an alternative that defers report formatting.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Submit a timeline for at least 100 ms, including event arrival, service start, service completion, and greatest observed response time in the simulation.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 04 / CORE READING

PDP-11 and MSP430: kinship and tradeoffs

Guiding question: Architectural resemblance reveals engineering tradeoffs

Key vocabulary: Addressing mode, instruction encoding, source/destination, constant generator

4.1 Architectural similarities that matter

The MSP430 has sixteen 16-bit register designators: R0/PC, R1/SP, R2/SR and special-purpose decoding, R3/constant generator, and R4–R15 as twelve general working registers. The PDP-11 has eight programmer-visible 16-bit registers, with PC and SP incorporated into the same general register numbering. Both machines permit many source/destination operations and address memory through register-based modes. The surface resemblance in assembly language is not an accident of spelling: the machines share powerful ideas about registers and operands.

However, one must be precise. The MSP430 is not binary-compatible with the PDP-11 and cannot execute PDP-11 machine code directly. Its source and destination addressing bits are asymmetric, its interrupt system and peripherals differ, and its registers have specialized encodings. TI documentation historically noted that the MSP430 can emulate much of the PDP-11 *instruction functionality*; that is not the same as running an unchanged PDP-11 executable.

4.2 A side-by-side example

PDP-11: `MOV #10,R4`, `ADD R4,R5`, `MOV (R5)+,R6`. MSP430: `MOV.W #10,R4`, `ADD.W R4,R5`, `MOV.W @R5+,R6`. The final line on both sides reads through a register-held pointer and advances that pointer. Syntax differs, and exact encoded instructions differ, but the conceptual mapping is unusually close.

Why does the MSP430 have twelve general working registers? Its 16-bit instruction encoding assigns four bits to a register number, allowing sixteen register choices. But those bits consume space. In the common two-operand format, the source has two mode bits while the destination has just one, so the destination cannot use every source addressing form. The PDP-11 uses three mode bits plus three register bits for either operand in many double-operand instructions. That symmetry yields richer addressing while leaving fewer register numbers.

4.3 The constant generator is a design lesson

MSP430 addressing encodings involving R2 and R3 can synthesize common constants such as 0, 1, 2, 4, 8, and -1 without fetching an additional immediate word in many circumstances. This reduces code space for frequent operations. Yet R2 and R3 are not interchangeable with general working registers; the same encoding fields acquire specialized meaning. Architectural elegance is always a balance between programmer convenience, instruction word capacity, hardware complexity, and code density.

Prediction exercise. Consider repeated operations adding 1 or subtracting 1. Why might an instruction set devote encoding shortcuts to such constants? What is the tradeoff in address-mode flexibility? Find two instructions in the MSP430 generated assembly whose constants are encoded without an immediate extension word; verify with a listing file, not by assumption.

4.4 RISC and CISC: useful words, unhelpful labels

The 2003 Jonathan Kirwan essay supplied in the course materials notes the PDP-11 resemblance and questions the MSP430's marketing classification as RISC. That is a worthwhile historical debate. Traditional load/store RISC designs separated memory access from arithmetic and often aimed for simpler instruction decoding and more regular

pipeline behavior. MSP430 instructions frequently allow memory operands and variable instruction lengths; TI nevertheless describes its processor family as a 16-bit RISC architecture. The value of the comparison lies in examining what the hardware actually permits, not awarding a categorical label.

Compare an MSP430 instruction that moves a word from memory with one that moves between registers. Assemble both. Inspect number of words, encoding fields, and documented cycle counts. Then measure where possible—but never confuse a GPIO pulse around a sequence with timing of one isolated instruction unless all overhead has been measured or subtracted.

4.5 Worked paper exercise: same algorithm, different syntax

Write a summation loop for 5 through 1. On the PDP-11, the `SOB` instruction can both decrement a register and branch. On the MSP430, use `DEC.W` and `JNZ` as a corresponding two-instruction idiom. Explain why semantically equivalent code may require differing instruction counts and cycles. Keep one table for arithmetic state and a separate table for instruction addresses and encodings.

CAP gate. Publish an annotated comparison with the exact CPU variant, assembler syntax, source listing, and a note explaining what has *not* been tested. No statement of common binary lineage is supported by this example.

Why four more register-select bits change the design

An instruction format is a physical budget. A sixteen-bit instruction word cannot simultaneously allocate unlimited opcode bits, register bits, mode bits, and size control. The PDP-11's three-bit register identifier selects one of eight registers; the MSP430's four-bit field selects one of sixteen. Two named operands can therefore consume more of the fixed instruction word on the MSP430.

The MSP430 preserves many elegant source forms while restricting destination addressing. R2 and R3 encodings offer selected constants and special addressing operations. These are not inelegant afterthoughts to be memorized; they are clues to the design pressure imposed by an economical instruction representation.

A learner should prove one encoding with the assembler listing instead of trusting this paragraph. Compile an immediate constant, an indexed operation, and a register-register ADD. Record the instruction words and extension words for each. Compare the source length with the stored machine-code length. That is a reproducible way to discuss code density without asserting that one architecture is universally superior.

Instruction encoding laboratory: where did the bits go?

The PDP-11 used three bits for each register identifier and three bits for each operand addressing mode. Its regular double-operand format made both operand fields six bits. In the MSP430 Format I representation, a four-bit source register field and two source-mode bits coexist with a four-bit destination register field, only one destination-mode bit, an opcode field, and a byte/word bit. The teaching question is not whether the later CPU has more registers but how the architectural design spends the limited space in its sixteen-bit instruction word.

For a controlled experiment assemble `MOV.W R4,R5`, `MOV.W @R4+,R5`, `MOV.W #8,R5`, and `MOV.W #1234,R5` using one known MSP430 assembler. Preserve the compiler/assembler command and generated listing. Predict which instructions use a constant-generator encoding and which need an additional word. Because the operands occupy different places in the instruction field, the source register and addressing mode affect the instruction word even when the mnemonic is the same.

Then switch to the PDP-11 and inspect the source syntax for register, indexed, and autoincrement operations. The PDP-11 also uses the PC as an addressable register, so immediate operands can be understood as a special use of PC autoincrement. Similar-looking operands do not always have identical restrictions or encoding, and source and destination modes must be verified against the individual architecture manuals.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

4.1 Explain why naming sixteen registers requires four bits and naming eight requires three; describe the effect on a two-operand instruction.

4.2 Compare `MOV.W #1,R5` and `ADD.W #1,R5`: do the operations have the same semantics, even if one encoding is shorter?

4.3 List the MSP430 special register functions of R0, R1, R2, and R3.

4.4 Write equivalent PDP-11 and MSP430 statements for register addition, indexed memory read, and source autoincrement.

4.5 Why does the PDP-11 allow more symmetric operand addressing than classic MSP430? Give the instruction-format explanation rather than a marketing label.

4.6 Explain why architectural resemblance is insufficient evidence of direct historical descent from DEC to TI.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/08_compare_register_loop.asm

```
1   ; TI-style MSP430 assembly fragment (not a standalone firmware image).
2   ; R4 counts down 5..1, R5 accumulates 15 decimal.
3           MOV.W   #5, R4
4           CLR.W   R5
5   loop:   ADD.W   R4, R5
6           DEC.W   R4
7           JNZ     loop
8   ; Place inside correct TI assembler source with reset/startup/linker vectors.
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 04: A Thousand Hertz Is a Claim to Test

Timer_A, calibrated DCO, independent Pico measurements

Industrial or historical context

The first laboratory on the real MSP430 builds a deliberately simple 1 kHz output on P1.6. A nominal 1 MHz SMCLK clocks Timer_A. Up mode counts from 0 to CCR0, inclusive. CCR0=499 means 500 timer ticks. When output mode 4 toggles TA0.1 once for each timer period, a complete square-wave cycle requires two toggles: the ideal frequency is 1,000,000/(2 × 500), or 1000 Hz. The hardware timer changes the pin while the CPU can remain in an idle loop. This distinction is essential: a timer-driven waveform is not a bit-banged software loop.

Model and technical reasoning

The frequency meter is a second system with its own timebase. Even if the target is nominally set to 1 MHz, actual drift and temperature can change the signal. The Pico counts rising edges during a measured gate. Capture both pulse count and elapsed duration, not only a formatted hertz number. A one-second gate gives roughly one-hertz count resolution before clock and gate errors are considered; apparent extra decimal digits do not establish matching accuracy. The duty cycle may depend on compare timing and output mode. Measure that separately when a pulse-width tool is available.

Prediction before measurement

Derive the expected frequency for CCR0 values 499, 999, and 249 before changing the program. Explain why CCR0=999 does not mean 999 timer ticks.

Procedure and evidence gates

1. Verify physical 3.3 V target power, proper pin mapping, debugger isolation, and shared signal ground.
2. Build the known Day One assembly example and preserve build/verification output.
3. On the Pico set generator OFF, input GP3, one sample, and a 1000 ms gate; read back STATUS?.
4. Record five separate captures at CCR0=499, including pulse count and actual elapsed microseconds.
5. Change only CCR0 to 999; rebuild and reflash. Collect a second dataset and compare with the prior prediction.

Interpretation questions

1. Why does the CPU’s main loop not set the edge spacing?
2. Could a 1000.000 reading be wrong?
3. What measurement would establish duty cycle?
4. How would you distinguish target clock error from meter clock error?

Required submission: Photograph or wiring sketch, instrument STATUS? transcript, program checksum, five actual frequency captures for each setting, and a table of predicted versus observed results.

WORKSHEET 04 / REPRODUCIBLE LAB RECORD

A Thousand Hertz Is a Claim to Test

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 04 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

A Thousand Hertz Is a Claim to Test • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: Why does the CPU's main loop not set the edge spacing?

_____

_____

Prompt: Could a 1000.000 reading be wrong?

_____

_____

Prompt: What measurement would establish duty cycle?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 04 / DESIGN AND REASONING

Measure instruction size rather than judging source size

Prepare MSP430 assembly fragments for a register-register operation, an immediate constant, a far larger immediate, and an indexed memory access. Use the actual assembler listing to count stored 16-bit words. Add a discussion of why the number of textual characters in the source does not measure program memory consumption.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Attach the exact assembler version, code listing, 16-bit opcodes, extension words, and a grounded explanation of any deviations from your prediction.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

INDUSTRIAL COMPUTING / FOUNDATIONS AND PRACTICE

Part II — The MSP430 becomes a physical experiment

HISTORY • ARCHITECTURE • MEASUREMENT • CONTROL

CHAPTER 05 / CORE READING

Build a one-kilohertz signal, then measure it

Guiding question: A predicted waveform becomes a measured event

Key vocabulary: DCO, SMCLK, Timer_A, compare, toggle, frequency counter, gate

5.1 The question and the apparatus

The first physical question is deliberately narrow: Can an MSP430G2553 produce a nominal 1,000-Hz square wave, and can our external instrument observe it? The learner needs only a few ideas: clock, timer count, compare operation, output pin, and frequency meter. Everything else is initially support infrastructure. The existing Day One guide uses a custom MSP430G2553 board, the MSP-EXP430G2ET as a debugging/programming bridge, and a Pico W meter with a Web Bluetooth dashboard.

The board map in the existing documentation identifies P1.6/TA0.1 at custom header J2 pin 7, connected to Pico GP3; common ground is essential. The G2ET debugger-side interface connects to the custom board through power, ground, RST/SBWTDIO and TEST/SBWTCK after appropriate isolation of the LaunchPad target. The board must have one verified target 3.3-V source. The Pico may use separate USB power, but its 3.3-V rail should not simply be tied to another active 3.3-V regulator. No 5-V signal should be connected to the MSP430 or Pico input.

These are *project-specific proposed connections*, not instructions to bypass the physical check. Before programming, identify printed board labels, orientation, target power, possible alternate-function conflicts, and whether the board is populated and electrically tested. Do not assume a routed KiCad design proves a safe physical device.

5.2 Derive the expected frequency

Start with a nominal 1,000,000-Hz SMCLK. Timer_A in up mode counts from zero through `CCR0` inclusive, giving `CCR0+1` timer ticks. Select `TA0CCR0 = 499`; the timer period is 500 microseconds. If output mode 4 toggles channel TA0.1 once per timer period at a compare point `CCR1`, a *full* square-wave period requires two toggles. Thus the nominal output frequency is `f_out = f_SMCLK / [2 × (CCR0+1)] = 1,000,000 / (2 × 500) = 1,000 Hz`.

A common mistake is to conclude that 500 ticks at 1 MHz produces a full 2-kHz square-wave cycle. It produces one **edge** every 500 microseconds in this configuration, not a full high-low-high cycle. A second common mistake is to treat exactly 1 MHz as a measured DCO frequency. Factory constants nominally configure a clock, but actual oscillator behavior depends on calibration conditions, supply, temperature, and time.

5.3 The code is a causal chain

The supplied candidate C program `code/msp430/01_timer_1khz.c` stops the watchdog, checks factory calibration, configures the DCO, selects P1.6's Timer_A function, writes two compare registers, and starts the timer. Its `for (;;)` loop does not generate edges: the peripheral operates independently while the CPU does nothing useful. That separation is the moment when the student begins to see a computer as a collection of concurrent hardware actors rather than a single stream of software instructions.

To align with the user's earlier work, repeat the experiment using the already built assembly source in the existing Day One project. Keep assembly and C as separate builds, not firmware concatenated into one image. Preserve compiler commands, generated listings, maps, binary/TI-TXT files, and target part numbers for both.

5.4 Measure honestly

The original Pico W meter configuration for the exercise requires its generator OFF, input GP3, one sample per reading, and a documented gate, such as 1,000 ms. The user's guide notes that saved Pico settings can override power-on defaults; read the live status before trusting the display. A one-second count near 1,000 edges can be informative, but integer-edge counting and the meter timebase limit absolute accuracy.

Take five distinct readings, with true observation times and instrument settings. Record frequency, edge count, elapsed microseconds, and any failed reads. Calculate minimum, maximum, mean, observed range, difference from the 1,000-Hz nominal target, and any reported decimal precision. Do not infer traceable calibration from the number of digits on the screen. A five-reading cluster near 1,000 Hz establishes repeatability under those conditions more readily than absolute accuracy.

5.5 The controlled-change test

Before changing code, predict what will happen when `TA0CCR0` changes from 499 to 999 while the compare is still within the count range. The expected nominal result is 500 Hz. Build and program, measure again, and compare the direction and magnitude of change. If the measured frequency increases, suspect a wrong assumption, a wrong pin, an old image, or instrument configuration before claiming an unexpected physical law.

CAP gate. Submit a wiring sketch, board and firmware revisions, five real captures, a clear derivation, one deliberate modification, and a statement distinguishing nominal design from measured outcome.

Instructor note. This lab was source-built in the earlier project, but a complete hardware bench result was not established in the source guide available to this edition. Mark that status until fresh evidence is saved.

Worked computation: count period is not square-wave period

Assume SMCLK is nominally 1 MHz and Timer_A counts through the sequence 0 to 499. That is 500 clocks per counting interval. If the selected output mode toggles the output once each interval, the time between toggles is 500 microseconds. The time between matching rising edges is two intervals, or 1,000 microseconds. The nominal frequency is therefore 1 kHz.

Now change CCR0 to 999 while keeping the same timer source. Each interval becomes 1,000 clocks; two intervals make an output cycle, and the expected output becomes 500 Hz. This controlled intervention is more informative than repeating the same measurement. It tests whether the observed circuit responds in the predicted direction and magnitude.

Never substitute nominal clock assumptions for evidence. If the Pico gate depends on its own uncharacterized crystal, a frequency disagreement can arise in either instrument. The first experiment is about generating and comparing a waveform. Calibrating either timebase is a different experiment and must be described separately.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

5.1 Derive the timer output frequency for SMCLK = 1,000,000 Hz and CCR0 = 249, 499, and 999, assuming one toggle each counting interval.

5.2 Explain why CCR0=499 means 500 timer ticks rather than 499.

5.3 If the meter observes 997 rising edges over 1.001 s, calculate the displayed frequency and signed departure from 1 kHz.

5.4 Name at least three ways a measured frequency can differ from the nominal mathematical prediction.

5.5 Explain why a one-second gate cannot directly prove a six-microsecond 1-Wire pulse width.

5.6 A signal frequency appears stable over five trials. Why might absolute accuracy still be unknown?

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/01_timer_1khz.c

```
/* 01_timer_1khz.c — Candidate source for MSP430G2553, TI/GCC msp430.h.
 * Requires target build, correct board wiring, and physical verification.
 * P1.6/TA0.1 -> Pico GP3; GND common; external supply 3.3V only.
 */
#include <msp430.h>

int main(void) {
    WDTCTL = WDTPW | WDTHOLD;
    if (CALBC1_1MHZ == 0xFF || CALDCO_1MHZ == 0xFF) {
        for (;;) { /* Calibration missing: STOP, don't erase InfoA. */ }
    }
    DCOCTL = 0;
    BCSCTL1 = CALBC1_1MHZ;
    DCOCTL = CALDCO_1MHZ;
    BCSCTL2 &= ~(SELS | DIVS_3); /* DCO source, SMCLK /1 */

    P1DIR |= BIT6;
    P1SEL |= BIT6;
    P1SEL2 &= ~BIT6;
    TA0CCR0 = 499;             /* Counts 0..499: 500 ticks */
    TA0CCR1 = 250;             /* Compare once per timer period */
    TA0CCTL1 = OUTMOD_4;      /* Toggle on CCR1 compare */
    TA0CTL = TASSEL_2 | MC_1 | TACLR; /* SMCLK, up mode */
    for (;;) { __no_operation(); }
}
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

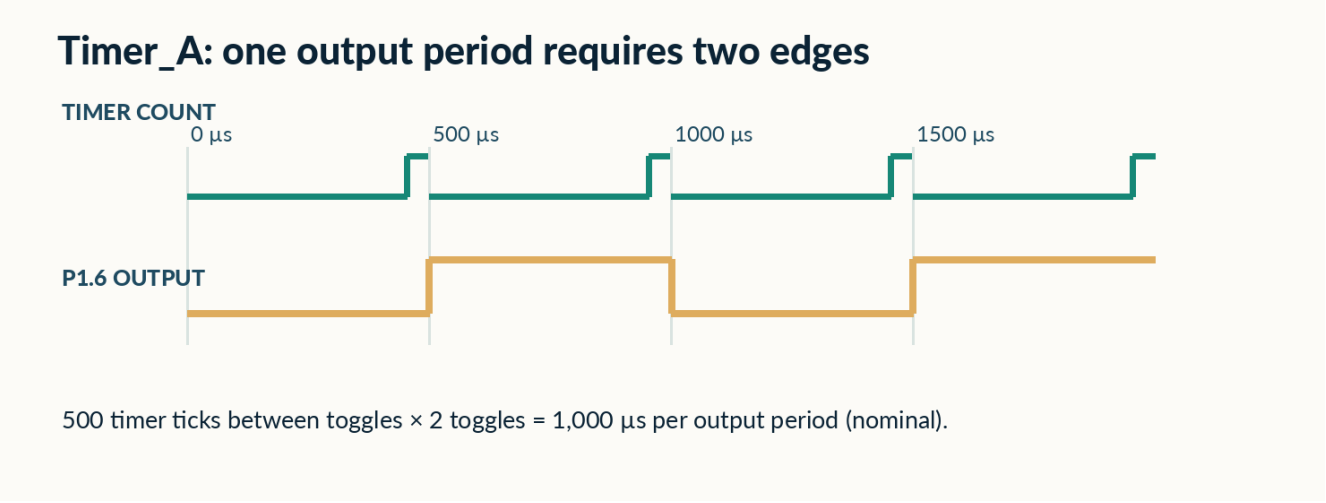


Figure 5.1 — Conceptual illustration for Chapter 5; not a verified wiring diagram.

Field laboratory 05: One Behavior, Two Programming Languages

Assembly, C, generated code, and compiler discipline

Industrial or historical context

A C statement that writes a timer register does not execute the same instructions in every toolchain. The compiler may use different instruction forms, initialize the stack, insert startup code, and choose an optimizer strategy. Yet the peripheral can still receive the same configuration and produce the same external signal. That is the right criterion for a controlled comparison: equivalent hardware state and observable behavior, not an identical count of assembly lines.

Model and technical reasoning

Place the original assembly Timer_A initialization beside code/msp430/01_timer_1khz.c. Identify the clock calibration constants, pin-function selection, CCR0, CCR1, output mode, and timer-start register. A key issue is whether an instruction is executed once during setup or repeatedly during output generation. A code-generation difference in setup may not change a hardware timer's steady-state frequency, while a difference in software-delay implementation can alter a bit-banged waveform dramatically.

Prediction before measurement

Before compiling the C version, predict which C statements will require memory writes and which may be optimized away. Explain why the `volatile` hardware register definitions matter.

Procedure and evidence gates

1. Capture the assembly baseline under an unchanged clock and meter configuration.
2. Compile the C candidate using a documented TI or msp430 GCC environment; save all warnings and linker output.
3. Review the reset vector, startup, register writes, and generated disassembly.
4. Flash only after checking target supply and pin compatibility; capture five new Pico gates.
5. Compare measured frequency, firmware source, toolchain version, and register-state equivalence.

Interpretation questions

1. Is a shorter assembly listing necessarily faster at the pin?
2. Why might a compiler initialize RAM before main?
3. What would change if the output were controlled by a delay loop instead of Timer_A?
4. Which differences constitute an invalid comparison?

Required submission: Build log, assembler and C source hashes, selected disassembly, hardware timer state, paired capture sheets, and an interpretation limited to the measured properties.

WORKSHEET 05 / REPRODUCIBLE LAB RECORD

One Behavior, Two Programming Languages

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 05 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

One Behavior, Two Programming Languages • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: Is a shorter assembly listing necessarily faster at the pin?

_____

_____

Prompt: Why might a compiler initialize RAM before main?

_____

_____

Prompt: What would change if the output were controlled by a delay loop instead of Timer_A?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 05 / DESIGN AND REASONING

Gate length and apparent precision

Repeat frequency counting at nominal 100 ms, 1000 ms, and 10,000 ms gates, where supported by the actual instrument. Compute edge count, elapsed time, measured frequency, and change between trials. Identify the quantization consequence of counting integral rising edges and how a longer observation can improve resolution without guaranteeing absolute accuracy.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Provide a table of real readings with the actual instrument status and a statement separating frequency resolution, precision, and timebase accuracy.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 06 / CORE READING

Memory-mapped I/O: from an instruction to a wire

Guiding question: One addressed bit changes a physical voltage

Key vocabulary: P1DIR, P1OUT, switch input, pull-up, GPIO polarity, volatile

6.1 What a register write really means

In C, a statement such as `P1DIR |= BIT0` looks like an ordinary expression. The left side represents a hardware register at a defined address. `BIT0` is a mask selecting one bit. The operator `|=` means read the current value, set that bit, and write the result back without intentionally changing the other bits. A write to a GPIO direction register can change the electrical function of a pin: input versus output. A write to an output register can change the output latch level.

The corresponding MSP430 assembly typically uses bit-oriented operations such as `BIS.B` and `BIC.B` with a mask and a direct hardware-register address. The PDP-11 had its own memory-mapped peripheral registers; the vocabulary of addressable device state carries across even when instruction encodings and buses do not. The student's mental model should always include the target memory address and the physical device beyond it.

6.2 An active-low switch

The project guide maps a switch to P2.0, with a connection to ground when pressed. To read this reliably, make the pin an input, enable its internal resistor, and select pull-up. The idle input should read 1 and the pressed input 0, assuming wiring matches the proposed netlist. This inversion surprises beginners until they draw the circuit. `P2IN & BIT0` is a masked hardware read, not a declaration that a button is 'true' or 'false' by itself.

The source `code/msp430/02_gpio_switch.c` reads the switch and commands P1.0. Check LED polarity in the actual board; logic 1 may not mean 'visibly illuminated' in every circuit. A voltmeter at the pin can distinguish program logic from an LED installed backward or a missing current-limiting component. Build each example separately, and do not reuse P1.6 for an LED if the frequency-meter lesson currently occupies it.

6.3 Volatile and changing external state

A hardware input can change between two processor instructions without any assignment in C. That is why device-register definitions carry volatile semantics and why an optimizing compiler must not blindly reuse a previously loaded value as though it were an ordinary local variable. `volatile` does not solve atomicity, protect access from interrupts, create a memory barrier for all possible hardware, or make an operation fast. It tells the compiler a narrower but essential truth about observable accesses.

Prediction exercise. Write out the eight bits of P1DIR before and after `P1DIR |= BIT0`, then after `P1DIR &= ~BIT0`. If a second unrelated output occupies BIT4, why is a direct `P1DIR = BIT0` assignment dangerous? What could happen if two interrupt contexts both modify a shared software variable, even though that variable is `volatile`?

6.4 Control versus observation

Attach a physical switch, then probe the GPIO output or LED with the Pico or a meter. Record input state and output state as distinct data. If you can only see the LED, you know less than if you can also measure voltage at the pin. If

the output is wrong, debug the chain in order: observed switch level, pin direction, pull-up, input mask, code branch, output latch, external circuit. Treat a failed test as evidence about one link in the chain.

CAP gate. Publish a truth table with real input/output states and a diagram that identifies electrical levels rather than only labels such as 'pressed' and 'on'.

Binary view of an active-low switch

An ordinary button can introduce three essential ideas at once: voltage is not a Boolean value, a register contains multiple independent bits, and physical polarity is a circuit property. With P2.0 configured as input and an enabled pull-up, an open switch typically reads as one. Closing the switch to ground pulls the pin low and makes the sampled bit zero.

The program may convert that raw bit to the semantic state “pressed.” The latter is an interpretation, not a voltage level. The statement `pressed = ((P2IN & BIT0) == 0);` is an explicit contract translating electrical behavior into a meaningful variable.

The motor-control analog is straightforward. A field input might be active low, normally closed, or isolated through an optocoupler. A developer who mistakes raw signal polarity for application meaning could reverse an alarm condition. Students should draw a truth table before claiming that an LED proves their software was correct.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

6.1 Show the binary effect of `P1DIR |= BIT0` when P1DIR initially equals 0b01010100.

6.2 For a switch tied to ground with a pull-up, write the raw P2.0 bit and logical pressed value in both switch states.

6.3 Explain why an LED may be OFF when the corresponding GPIO output latch is high.

6.4 Write a pseudocode debouncer and define its sampling interval and acceptance rule.

6.5 Contrast a memory-mapped register write with writing to ordinary RAM at the same source-language syntax level.

6.6 Propose one physical measurement that can refute an incorrect GPIO schematic assumption.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/02_gpio_switch.c

```
 1   /* 02_gpio_switch.c — Active-low switch P2.0 controls LED on P1.0.
 2    * Verify physical polarity; a P1.0 high may or may not light your LED.
 3    */
 4   #include <msp430.h>
 5   int main(void) {
 6       WDTCTL = WDTPW | WDTHOLD;
 7       P1SEL &= ~BIT0; P1SEL2 &= ~BIT0;
 8       P1DIR |= BIT0; P1OUT &= ~BIT0;
 9       P2SEL &= ~BIT0; P2SEL2 &= ~BIT0;
10       P2DIR &= ~BIT0; P2REN |= BIT0; P2OUT |= BIT0;
11       for (;;) {
12           if ((P2IN & BIT0) == 0) P1OUT |= BIT0;
13           else P1OUT &= ~BIT0;
14       }
```

```
15    }
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 06: A Bit Reaches the Outside World

Memory-mapped I/O, polarity, and switch input

Industrial or historical context

Memory-mapped I/O is the concrete bridge between the PDP-11 and the MSP430. A memory address can represent a physical interface register, not merely ordinary RAM. Writing a one to a GPIO direction register changes a pin into an output; clearing the corresponding bit changes it back into an input. Writing to an output latch may not light an LED if the signal is inverted by the board wiring, a transistor, or an active-low circuit. Engineering begins by checking the schematic and the pin voltage.

Model and technical reasoning

Use the code/msp430/02_gpio_switch.c example. Configure P1.0 as an output and P2.0 as an input with a pull-up. A button connected to ground makes a pressed state read as logic zero. Students must draw both the physical circuit and the binary value in the input register; the false intuition that pressed always means one should be explicitly challenged. On an unverified custom board, a safe logic analyzer or voltmeter comes before assumptions about the indicator lamp.

Prediction before measurement

For a P2.0 switch that shorts the line to ground when pressed, predict P2IN & BIT0 in the pressed and released states. Write the binary mask and show how P1DIR changes.

Procedure and evidence gates

1. Confirm P1.0 LED polarity and P2.0 button-to-ground connection on the actual board.
2. Compile and flash the GPIO candidate source without combining it with the Timer_A experiment.
3. Measure P2.0 idle voltage and pressed voltage, and record both digital input states.
4. Observe the LED or measured P1.0 output under each button condition.
5. Disconnect power; describe what additional protection would be needed for a real industrial switch input.

Interpretation questions

1. What does `P1OUT |= BIT0` preserve?
2. Why can contact bounce matter even if the LED looks steady?
3. Why is an internal pull-up not a general solution for industrial field wiring?
4. What can a measured 3.3 V pin tell you, and what can it not tell you?

Required submission: Switch truth table with measured volts, GPIO-register snapshot or code trace, LED/pin results, and a note on wiring-dependent polarity.

WORKSHEET 06 / REPRODUCIBLE LAB RECORD

A Bit Reaches the Outside World

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 06 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

A Bit Reaches the Outside World • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: What does `P1OUT |= BIT0` preserve?

_____

_____

Prompt: Why can contact bounce matter even if the LED looks steady?

_____

_____

Prompt: Why is an internal pull-up not a general solution for industrial field wiring?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 06 / DESIGN AND REASONING

Debounce an unreliable contact

Use an external input record or a simulated bounce sequence to compare two debounce rules: fixed quiet interval and consecutive matching samples. Specify the sampling period and acceptance count before observing the result. Ensure that either rule has a bounded response delay. Identify the tradeoff between latency and false triggers.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Publish a synthetic or measured timestamped edge series, the output of both algorithms, and a calculation of worst-case debounce delay.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 07 / CORE READING

Real time is a promise, not a clock speed

Guiding question: Deadlines matter more than average speed

Key vocabulary: Latency, interrupt service routine, jitter, timer, masking

7.1 When an external event interrupts a calculation

Imagine a filling station. A sensor reports that a container has reached its target level. The controller must close a valve before the contents overflow. The correctness of the action depends on two dimensions: the valve closes, and the command arrives in time. A beautifully structured program that misses the deadline is not a correct real-time program. This distinction was central to the operating environments developed for the PDP-11, particularly RSX-11M and RSX-11S, and it remains central to the MSP430.

A novice program might repeatedly inspect a switch: `while (1) if (switch_pressed()) close_valve();`. Polling can be appropriate when the interval between polls is bounded and the application tolerates it. But the loop can be delayed by display formatting, serial transmission, or unexpectedly slow arithmetic. Interrupts provide another mechanism: when an event occurs, dedicated hardware redirects execution to a service routine. Neither mechanism is magically deterministic. We must account for interrupt masking, instruction completion, nested interrupts, resource sharing, and the time consumed by the ISR.

A PDP-11 installation might combine interrupts, device controllers, and RSX task priorities. The MSP430 has no RSX executive in this example; instead, its vector table, peripheral interrupt flags, status register, and a small main loop accomplish the necessary coordination. We should teach the *responsibility* that survives across both systems rather than pretending their implementations are the same.

7.2 Three kinds of timing

Elapsed time is the physical interval measured between events. **CPU time** is the interval spent executing instructions. **Response latency** is the time between an external event and the intended reaction. The first two are not enough to establish the third. In addition, **jitter** is variation in observed timing from one instance to another. A control event can be timely on average and still occasionally miss a deadline.

For the 1-MHz clock used in the introductory lab, a cycle is nominally 1 microsecond. But it is incorrect to conclude that an arbitrary C statement takes 1 microsecond. Instructions differ in cycle counts, access modes, compiler output, and effects on interrupts. Instead, use a timer-based stimulus and a second GPIO pin as an observation point; measure the separation between events using a pulse-width-capable instrument. Keep the CPU clock assumption separate from the observation instrument's clock uncertainty.

7.3 Interrupt exercise: a periodically changing output

In `code/msp430/03_interrupt_led.c`, Timer1_A is configured with nominal SMCLK 1 MHz divided by four. With `TA1CCR0 = 24999`, the 25,000-count interval is nominally 100 milliseconds, and a Timer_A interrupt toggles P1.0. A full high-low-high LED output cycle therefore takes nominally 200 milliseconds. This experiment contrasts with Chapter 5, where Timer_A output hardware toggles a pin without CPU intervention.

CODE EXAMPLE · in-text excerpt

```
TA1CCR0  = 24999;                     /* 100 ms at 250 kHz */
TA1CCTL0 = CCIE;                      /* interrupt on CCR0 */
TA1CTL   = TASSEL_2 | ID_2 | MC_1 | TACLR;
__bis_SR_register(LPM0_bits | GIE);
```

```
#pragma vector=TIMER1_A0_VECTOR
__interrupt void timer1_a0_isr(void) {
    P1OUT ^= BIT0;                      /* visible toggling */
}
```

This excerpt needs the complete initialization and compiler support found in the companion candidate. A useful advanced variant is to toggle a **spare verified GPIO** on ISR entry and exit. The measured pulse width then estimates ISR execution time plus GPIO overhead; a second stimulus or marker is needed to measure interrupt latency. Explain the difference before presenting any timing plot. Do not claim a one-microsecond response simply because SMCLK is near 1 MHz.

7.4 Design exercise: a deadline budget

Suppose a hypothetical process requires response within 5 milliseconds. Allocate 1.0 ms for sampling, 0.5 ms for computation, 0.5 ms for output update, and 1.0 ms for worst-case waiting. The sum is 3.0 ms, leaving 2.0 ms of illustrative margin. Each number must eventually be replaced with measured or justified maxima. If the process has a relay that takes 20 ms to release, the software budget alone is irrelevant; actuator physics must be included.

CAP gate. Show a measured interval distribution with a defined start event and end event, and report the maximum seen—not just the mean. State what would be required to establish a true worst-case bound.

Jitter: compare two ways to move the same pin

Create one signal with a hardware timer compare output and another with an interrupt service routine that toggles a GPIO bit. Both may have similar average frequency. They are not equivalent instruments for precise edge placement. The hardware output-compare edge is tied to timer hardware; the software edge is delayed by interrupt recognition, entry, other interrupts, and the instructions preceding the port write.

Capture many edges with a timing-capable instrument. For each adjacent pair, compute the interval. Report the minimum, maximum, and range as well as the mean. If the measurement resolution is coarse, say so. An attractive average cannot conceal unstable edges.

This is also a practical bridge back to RSX-11M. The software problem of selecting a task and the electrical problem of meeting a deadline are related, but not identical. Priority helps organize response, while measurement demonstrates whether a particular built system meets its declared timing budget.

A concrete deadline budget

Consider an event at an MSP430 input. Before useful application code executes, the signal must reach the pin and meet electrical requirements, the interrupt mechanism must recognize it, the CPU must complete or interrupt its current activity as defined by the architecture, and context must be saved. The handler then performs instructions and may wake lower-priority code. If a handler toggles a measurement pin only after inspecting a flag, that inspection is part of the observed latency.

Write a budget using illustrative numbers: input recognition 5 microseconds, entry and context 8 microseconds, handler work 20 microseconds, and exit or handoff 7 microseconds. The nominal sum is 40 microseconds. It is only a paper model: real intervals must be obtained from the target clock, interrupt settings, compiler-generated code, and a timing instrument. If another operation can postpone interrupts by 70 microseconds, the model must include that delay in its worst-case bound.

Students should capture at least twenty time intervals rather than reporting a single convenient one. Record not only an average but minimum, maximum, the number of samples, and the instrument time resolution. A measured maximum over twenty events is evidence about those events; it is not a proof of the absolute worst case under all legal operating conditions.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

7.1 Compare the expected edge-to-edge timing of a Timer_A compare output and an ISR-driven GPIO toggle at the same nominal rate.

7.2 A periodic task runs every 20 ms and requires 1.5 ms. Calculate its ideal CPU utilization and discuss two limitations of that figure.

7.3 Define response time from physical input transition to output change; list the layers through which delay accumulates.

7.4 Describe the cost and risk of disabling interrupts around a timing-critical 1-Wire slot.

7.5 Provide a reason a polling loop can be preferable to interrupts in a narrow dedicated controller.

7.6 Design a repeatable test that reports the worst of 100 measured latencies.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/03_interrupt_led.c

```
 1  /* 03_interrupt_led.c — P1.0 toggles every 100ms, nominally.
 2   * TA1 in up mode from nominal 1MHz SMCLK/4 = 250kHz.
 3   * This is a distinct firmware target; don't combine with others unreviewed.
 4   */
 5  #include <msp430.h>
 6  int main(void) {
 7      WDTCTL = WDTPW | WDTHOLD;
 8      if (CALBC1_1MHZ == 0xFF || CALDCO_1MHZ == 0xFF) for (;;) {}
 9      DCOCTL = 0; BCSCTL1 = CALBC1_1MHZ; DCOCTL = CALDCO_1MHZ;
10      BCSCTL2 &= ~(SELS | DIVS_3);
11      P1SEL &= ~BIT0; P1SEL2 &= ~BIT0;
12      P1DIR |= BIT0; P1OUT &= ~BIT0;
13      TA1CCR0 = 24999;
14      TA1CCTL0 = CCIE;
15      TA1CTL = TASSEL_2 | ID_2 | MC_1 | TACLR;
16      __bis_SR_register(LPM0_bits | GIE);
17      for (;;) {}
18  }
19  #pragma vector=TIMER1_A0_VECTOR
20  __interrupt void timer1_a0_isr(void) {
21      P1OUT ^= BIT0;
22  }
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 07: When Time Becomes a Requirement

Interrupts, polling, jitter, and scheduling

Industrial or historical context

An industrial control system cares about deadlines, not just average execution speed. A 1 ms sensor event may be routine in one process and dangerously late in another. The PDP-11 family used priority interrupts and real-time software environments to manage multiple device demands. On the MSP430 a timer interrupt provides a small, accessible laboratory for understanding the same fundamental problem: the program is asked to suspend ordinary work, save enough state, perform a bounded action, and return.

Model and technical reasoning

Code/msp430/03_interrupt_led.c configures Timer1_A to interrupt on a 100 ms nominal interval. The ISR toggles an LED latch and returns. The waveform is distinct from the hardware Timer_A compare output. Delays from interrupt masking, competing code, or other interrupts can create jitter in a software-driven edge; a hardware output compare can be a better option when edge timing is the essential requirement. An interrupt provides responsiveness but does not magically guarantee a deadline.

Prediction before measurement

With a 250 kHz Timer1_A clock and CCR0=24999, calculate the nominal interval between ISR entries. How long is a complete cycle if the ISR toggles once per entry?

Procedure and evidence gates

1. Draw the timeline: timer period, interrupt request, CPU entry, GPIO write, return.
2. Build the ISR example; confirm actual interrupt vector and CPU/compiler conventions.
3. Observe P1.0 toggles. If supported, time-stamp edges using a suitable capture tool.
4. Add a short bounded foreground computation; measure whether observed timing changes.
5. Discuss a deliberately hypothetical case where ISR code takes longer than its scheduled period.

Interpretation questions

1. What is the difference between interrupt latency and interrupt execution time?
2. Why should an ISR avoid expensive printing?
3. When is polling preferable to interrupts?
4. How might the PDP-11's priority scheme affect lower-priority industrial tasks?

Required submission: Clock and deadline calculation, firmware version, GPIO timing trace, source explanation, and bounded assumptions about worst-case response.

WORKSHEET 07 / REPRODUCIBLE LAB RECORD

When Time Becomes a Requirement

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 07 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

When Time Becomes a Requirement • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: What is the difference between interrupt latency and interrupt execution time?

_____

_____

Prompt: Why should an ISR avoid expensive printing?

_____

_____

Prompt: When is polling preferable to interrupts?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 07 / DESIGN AND REASONING

A histogram of interrupt latency

Use an electrically observable stimulus, a controlled ISR output action, and a timing instrument capable of measuring their separation. Record 100 response intervals if equipment permits. Compute minimum, median, maximum, and 99th-percentile estimate with the sample size clearly stated. Compare this with a hardware timer compare output, ensuring the two tests use appropriate definitions of response.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Publish the raw interval record and a bounded statement about the largest latency observed, without claiming proof of an absolute worst case.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 08 / CORE READING

A machine that can report what it does

Guiding question: The controller explains itself through a serial link

Key vocabulary: UART, framing, baud, protocol, parsing, error status

8.1 From front panel to serial telemetry

A PDP-11 system could communicate through terminals, device interfaces, and connected peripherals. The ability to report state was essential in a plant: an operator needed to distinguish a command from an actual sensor reading and an alarm from a missing measurement. The MSP430G2553's USCI_A0 UART makes the same lesson accessible on a small board. It converts bytes into a start bit, data bits, and stop bit on a wire. The PC does not need to guess whether a line was printed: a terminal can show what arrived.

The user's custom-board guide identifies J3 as the 3.3-V TTL serial interface, with MSP430 P1.2 as TX and P1.1 as RX. Verify the board in hand. Connect target TX to adapter RX and common ground. Do not connect a legacy +/-12 V RS-232 port directly to the microcontroller. If the adapter has a 3.3-V power pin, leave that power pin disconnected unless a single-target-supply plan deliberately uses it.

8.2 An initial transmitter

The candidate `code/msp430/04_uart_hello.c` configures a nominal 1-MHz SMCLK, selects the USCI peripheral function on P1.1 and P1.2, and sets a nominal 9600-baud divider. It sends a known status sentence. The first lab should transmit a single `A`, inspect the terminal's 8N1 settings, and establish that every byte is correctly received before adding decimal formatting or periodic reports.

CODE EXAMPLE · in-text excerpt

```
static void putc_uart(char c) {
    while (!(IFG2 & UCA0TXIFG)) { }   /* await TX buffer ready */
    UCA0TXBUF = (unsigned char)c;
}
static void puts_uart(const char *p) {
    while (*p) putc_uart(*p++);
}
```

The loop waits for transmit-buffer availability; it does **not** guarantee that all bits of the preceding character have left the physical pin at that instant. For a known baud and frame size, estimate serialization time: an 8N1 frame contains ten bit times, so 9600 baud takes approximately 1.04 ms per character. A long print operation can dominate a tight loop. That is why industrial control should avoid placing blocking report functions inside time-critical interrupt routines.

8.3 Design a telemetry contract

Use short, explicit text records such as `T,123456,25.0625,77.1125,OK` for a timestamp in milliseconds, Celsius, Fahrenheit, and status. **That string is illustrative, not a measured value.** Prefer an unambiguous parser contract that also includes firmware version and sensor validity, for example `TEMP,v1,seq=12,raw=401,status=OK` where raw units are 1/16 degree Celsius. Host software can perform the engineering-unit conversion. This reduces formatting code in an MSP430 with limited RAM and flash.

Do not invent a timestamp source. If the MCU provides only a sample counter, label it `sample_id`, not Unix time. If a host later supplies reception timestamps, distinguish them from sample-acquisition timestamps; serial latency and clock offsets make these different observations.

CAP gate. Capture a real terminal transcript, a pin map, UART settings, clock source, and one known sequence of bytes. Deliberately change the host baud rate and explain the corruption as a mismatched communication contract rather than an inaccurate temperature sensor.

From characters to records

Start with one known character. A terminal that displays “A” demonstrates a useful first end-to-end path: an MCU loaded a transmit register, a serial peripheral framed the bits, an electrical link carried them, and a receiving tool decoded them under matching baud and parity settings. It does not show that every possible message or every power-up sequence is reliable.

Add a small structured record with explicit field names. A record such as `TEMP_C,25.0625,VALID,seq=17` is human readable, but its formal contract should also specify newline termination, units, allowed ranges, checksum policy if needed, and how missing records are detected. A sequence number exposes dropouts; timestamps require a defined clock source.

The telemetry path must not own the safety-related control decision. The MSP430 should remain capable of selecting its own safe output when the PC is detached. This separation is a modern miniature of the industrial distinction between a real-time control processor and a remote operator terminal.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

8.1 Sketch the start, eight data, and stop bits of a typical 8N1 UART frame; how many line bit periods does one byte occupy?

8.2 At 9600 bit/s, compute approximate duration of a 10-bit 8N1 frame.

8.3 What happens when MCU and terminal disagree on baud or voltage level? Distinguish ±12 V RS-232 from 3.3 V UART.

8.4 Design a telemetry row including sequence number, raw temperature, validity, and a defined unit.

8.5 Explain why an observer PC must not be required to keep the embedded supervisor within its operating policy.

8.6 List what a terminal transcript establishes and what it cannot establish about electrical signal quality.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/04_uart_hello.c

```
/* 04_uart_hello.c — TI/GCC MSP430G2553 at nominal 1MHz, ~9600 8N1.
 * J3: P1.2 TX -> 3.3V adapter RX; GND shared. Do NOT cross power rails.
 * Check actual baud and clock before use as an instrument.
 */
#include <msp430.h>
static void putc_uart(char c) {
    while (!(IFG2 & UCA0TXIFG)) {}
    UCA0TXBUF = (unsigned char)c;
}
static void puts_uart(const char *s) { while (*s) putc_uart(*s++); }
int main(void) {
```

```
12      WDTCTL = WDTPW | WDTHOLD;
13      if (CALBC1_1MHZ == 0xFF || CALDCO_1MHZ == 0xFF) for (;;) {}
14      DCOCTL = 0; BCSCTL1 = CALBC1_1MHZ; DCOCTL = CALDCO_1MHZ;
15      BCSCTL2 &= ~(SELS | DIVS_3);
16      P1SEL |= BIT1 | BIT2; P1SEL2 |= BIT1 | BIT2;
17      UCA0CTL1 |= UCSWRST;
18      UCA0CTL1 = UCSWRST | UCSSEL_2; /* SMCLK */
19      UCA0BR0 = 104; UCA0BR1 = 0; UCA0MCTL = UCBRS0;
20      UCA0CTL1 &= ~UCSWRST;
21      for (;;) {
22          puts_uart("STATUS,HELLO,1MHz_nominal\r\n");
23          __delay_cycles(1000000);  /* nominal ~1 sec, CPU-dependent */
24      }
25  }
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 08: Teaching a Machine to Report

UART framing, telemetry, versioned data contracts

Industrial or historical context

An industrial controller requires a way to expose its state to an operator, data historian, or diagnostic computer. A terminal connection on a PDP-11 served one part of this purpose; a small MCU serial port serves the same abstract purpose today. The controller must still function if the connection disappears. Separating the telemetry channel from the real-time decision loop is an important design habit. It also makes the experimental record more trustworthy: students should know which fields were measured, calculated, guessed, or defaulted.

Model and technical reasoning

The MSP430G2553 USCI UART can send bytes from P1.2 at an appropriate baud configuration. At nominal 1 MHz, the candidate example uses a baud divider for approximately 9600 bps. UART data consists of individual frames, typically one start bit, eight data bits, and a stop bit. A UART character can therefore take roughly ten bit times even before software formatting. Recording a transmission on a logic analyzer gives independent evidence of the framing and bit period.

Prediction before measurement

Compute the approximate on-wire time of a 30-byte message at 9600 baud in 8N1 format. Decide whether sending that message in a 100-µs critical section would be acceptable.

Procedure and evidence gates

1. Inspect the board J3 header and verify TTL-level 3.3 V signal compatibility.
2. Connect MCU TX to adapter RX and share ground; do not connect two active power sources.
3. Compile the UART hello example and capture the literal transmitted bytes in a terminal.
4. Check baud and framing with an independent tool or two agreeing configurations.
5. Define a CSV-style record with version, sequence, sensor status, raw temperature, controller state, and error code.

Interpretation questions

1. What information is lost when a message contains only a formatted temperature?
2. How should a receiver handle a truncated message?
3. Why is printf size relevant on a microcontroller?
4. What does the host need to do when a sequence number is missing?

Required submission: Actual serial transcript, wiring map, baud assumption and calculation, record schema, and one intentionally malformed-record handling test.

WORKSHEET 08 / REPRODUCIBLE LAB RECORD

Teaching a Machine to Report

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 08 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

Teaching a Machine to Report • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: What information is lost when a message contains only a formatted temperature?

_____

_____

Prompt: How should a receiver handle a truncated message?

_____

_____

Prompt: Why is printf size relevant on a microcontroller?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 08 / DESIGN AND REASONING

Find an intentionally damaged serial record

Build a host-side parser for a sequence-numbered telemetry line. Feed it correctly framed input, a missing field, an invalid unit, a repeated sequence number, and a malformed character. Define whether a bad record is rejected, flagged, or recovered and show that a dropped UART message does not alter the local controller state.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Submit parser tests, an example accepted record, four rejected records, and a short interface contract.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 09 / CORE READING

One wire, two participants, and a microsecond-scale contract

Guiding question: An electrical bus is a timed conversation

Key vocabulary: 1-Wire, release, pull-up, reset, presence, slot, rise time

9.1 The discipline of releasing a line

The Dallas/Maxim/Analog Devices 1-Wire protocol combines a physical electrical rule with a timed conversation. The master and device may pull the data line low. Under normal externally powered operation, neither should force the shared line high against the other; instead, the pull-up restores the high level after release. The distinction matters because the microcontroller output latch and output-direction register can cause an electrical conflict if the pin is inadvertently configured as a driven high output.

For this curriculum, use a genuine, externally powered DS18B20, with its VDD and ground correctly connected and a verified data pull-up to 3.3 V. The existing board's P1.7 net is a plausible location because the prior netlist assigns it an approximately 4.7-kΩ pull-up, but this must be confirmed on the actual board and checked for I²C conflicts. The sensor package pinout must come from its manufacturer datasheet, not from a generic drawing.

9.2 The primitive operations

A master needs only a few physical primitives: reset, sense presence, write one bit, and read one bit. Byte operations then build on bit operations, least-significant-bit first. The conceptual state transitions are:

CODE EXAMPLE · in-text excerpt

```
IDLE high -> MASTER LOW -> MASTER RELEASE -> DEVICE RESPONSE -> IDLE high
```

The reset interval and the read/write slots each have bounds specified in the datasheet. The critical time is the voltage transition observed *at the pin*. A C delay loop that says `wait_us(6)` is merely a hypothesis about the signal. Compiler optimization, overhead, interrupts, clock frequency, and pin capacitance can change what actually occurs. The physical test should show reset low time, release interval, presence response, a write-1 slot, a write-0 slot, and the read sampling point.

The optional candidate `code/msp430/09_onewire_transport_CANDIDATE.c` shows a Timer_A-based strategy, but it is intentionally **not presented as validated target firmware** in this edition. Its Timer_A ownership conflicts with the 1-kHz experiment, and its microsecond busy-wait and interrupt effects need independent bench evidence. The companion `code/msp430/07_onewire_transport_design.txt` is the deliberate interface contract for implementing and verifying transport in a separate target project.

9.3 Layer the driver; do not write everything at once

First construct and verify `ow_drive_low()` and `ow_release()` with a voltmeter or scope. Second verify reset/presence without sending any sensor command. Third verify one write-0, one write-1, and one read slot, including interrupt timing assumptions. Fourth implement LSB-first byte transmission and reception. Fifth read the sensor's 64-bit ROM identity or use Skip ROM only when exactly one device is attached. Finally command a conversion and read a nine-byte scratchpad.

For the DS18B20 at the factory-default 12-bit resolution, a conversion may require up to 750 ms; do not assume it is complete because 100 ms elapsed. Externally powered mode avoids the extra strong-pull-up complexity associated

with parasite power. Use a valid completion condition or a sufficient documented wait. Preserve raw bytes, the CRC result, and the status of every failed transaction.

9.4 A measurement contract for the Pico

A frequency meter that counts rising edges over one second is excellent for a thousand-hertz signal but insufficient as evidence for a six-microsecond 1-Wire pulse. For the 1-Wire laboratory, configure a separate validated pulse-width capture or logic-analyzer program. The measurement instrument's edge timing resolution, input threshold, buffer capacity, reference timebase, and acquisition trigger should appear in the report. A GPS-disciplined timebase can be a later experiment, not a condition for beginning.

CAP gate. Show at least one annotated physical reset/presence waveform and one of each bit-slot type. Include the independent pin-level measurement, timer configuration, interrupt policy, pull-up value, and device power mode.

Why the 1-Wire signal is not ordinary serial data

A 1-Wire device and its master cooperate electrically on one pulled-up data conductor. A normal idle is high because a resistor supplies current, not because every participant drives the wire high. To start a low pulse, the master configures an output that sinks current. To release the line, the master stops driving it low, and the pull-up returns it high.

This electrical convention matters to firmware: a line that remains actively driven high cannot safely coexist with a device attempting to signal low. The driver must separate “drive low” from “release.” A reset/presence exchange is consequently a time sequence of ownership changes, not simply a UART byte.

Measure the reset-low interval, the sensor response window, and representative write-one, write-zero, and read slots at the physical pin. Do not infer microsecond validity from observing a successful high-level temperature conversion. A 1 kHz frequency counter is useful elsewhere, but an edge-time capture or scope is necessary for the 1-Wire timing claim.

Engineering the one-wire port rather than copying a delay loop

Start with the electrical rule: a bus master must pull DQ low or release it. The master should never drive DQ actively high when the sensor may also drive it low. For the proposed MSP430 board the candidate DQ net is P1.7; inspect the physical pull-up and remove other active devices from that shared I/O line before applying voltage. Keep sensor VDD connected to 3.3 V for the introductory project; parasite-power mode introduces different power-delivery conditions.

The candidate 1-Wire transport uses Timer_A0 in continuous mode as a nominal microsecond counter. That imposes a real resource conflict: it cannot simultaneously use the same timer configuration as the earlier Timer_A0 1 kHz exercise. The exercises are intentionally separate firmware images. An engineer may later choose another timer or a dedicated PIO probe, but must document the revised ownership and routing.

For a write-one slot, a short low pulse is followed by release; for a write-zero slot, the low interval occupies most of the slot. A read slot begins with a low pulse, a release, and a sample taken during the device's defined response window. Show those phases as separate regions on a measured waveform. A logic analyzer used at an inadequate sample rate can make a nominal 6-microsecond pulse appear to have different widths, so characterize the capture instrument too. Bus capacitance, the resistor, wiring length, and the sensor's input characteristics all affect rise time.

Before trusting a temperature transaction, demand a reset-low capture, a presence response with plausible timing, a write-one and write-zero capture, and a read slot with documented sample location. Repeat after changing only the clock setting, then explain why a fixed software delay can fail when the DCO changes. Do not interpret a correct 1 kHz frequency reading as proof that these microsecond slots satisfy the DS18B20 datasheet.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

9.1 Draw the three distinct bus actions: drive low, release, and observe; mark the participant responsible for each.

9.2 Why can a pin that actively drives high be unsafe on a shared 1-Wire bus?

9.3 Explain how pull-up resistance and bus capacitance can affect valid logic timing.

9.4 Identify the separate evidence needed for reset, presence, write-one, write-zero, and read slots.

9.5 A library assumes an 8 MHz CPU while a project uses 1 MHz. Why is modifying a single delay constant insufficient to certify timing?

9.6 Distinguish a meter that measures average frequency from one that measures pulse width and edge timestamps.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/09_onewire_transport_CANDIDATE.c

```
/* MSP430G2553 1-Wire timing demonstration on P1.7.
 * IMPORTANT: Candidate source, not bench-verified. Must measure real slots.
 * Requires independently confirmed external ~4.7k pull-up to target 3.3 V.
 * Timer_A0 is dedicated to this example and therefore cannot simultaneously
 * generate the preceding 1 kHz experiment without redesign.
 * Timing is nominal assuming SMCLK calibrated near 1 MHz.
 */
#include <msp430.h>
#include <stdint.h>
#define OW BIT7

static void ow_low(void)
{
    P1OUT &= (unsigned char)~OW;  /* output latch first: never drive high */
    P1DIR |= OW;                  /* pull bus LOW */
}
static void ow_release(void)
{
    P1DIR &= (unsigned char)~OW;  /* input/high impedance; pull-up raises */
}
static void wait_us(unsigned int us)
{
    unsigned int start = TA0R;
    while ((unsigned int)(TA0R - start) < us) { }
}
void ow_init(void)
{
    P1SEL &= (unsigned char)~OW;
    P1SEL2 &= (unsigned char)~OW;
    ow_release();
    /* assumes prior 1 MHz DCO configuration */
    TA0CTL = TASSEL_2 | MC_2 | TACLR; /* continuous timer */
}
/* Returns 1 if presence was observed; 0 for missing/stuck bus.
 * Blocking; interrupts excluded for critical timing, investigate latency.
 */
unsigned int ow_reset(void)
{
    unsigned int present;
    unsigned int gie_was_set = __get_SR_register() & GIE;
    if ((P1IN & OW) == 0) return 0; /* idle must be high */
    __disable_interrupt();
    ow_low();     wait_us(480);
```

```
44      ow_release(); wait_us(70);
45      present = ((P1IN & OW) == 0);
46      wait_us(410);
47      if (gie_was_set) __enable_interrupt();
48      return present;
49  }
50  void ow_write_bit(unsigned int bit)
51  {
52      unsigned int gie_was_set = __get_SR_register() & GIE;
53      __disable_interrupt();
54      ow_low();
55      if (bit) { wait_us(6);  ow_release(); wait_us(64); }
56      else     { wait_us(60); ow_release(); wait_us(10); }
57      if (gie_was_set) __enable_interrupt();
58  }
59  unsigned int ow_read_bit(void)
60  {
61      unsigned int bit;
62      unsigned int gie_was_set = __get_SR_register() & GIE;
63      __disable_interrupt();
64      ow_low(); wait_us(6); ow_release();
65      wait_us(9); bit = ((P1IN & OW) != 0);
66      wait_us(55);
67      if (gie_was_set) __enable_interrupt();
68      return bit;
69  }
70  void ow_write_byte(uint8_t value)
71  {
72      unsigned int i;
73      for (i=0; i<8; ++i) { ow_write_bit(value & 1u); value >>= 1; }
74  }
75  uint8_t ow_read_byte(void)
76  {
77      unsigned int i;
78      uint8_t value=0;
79      for (i=0; i<8; ++i) if (ow_read_bit()) value |= (uint8_t)(1u<<i);
80      return value;
81  }
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

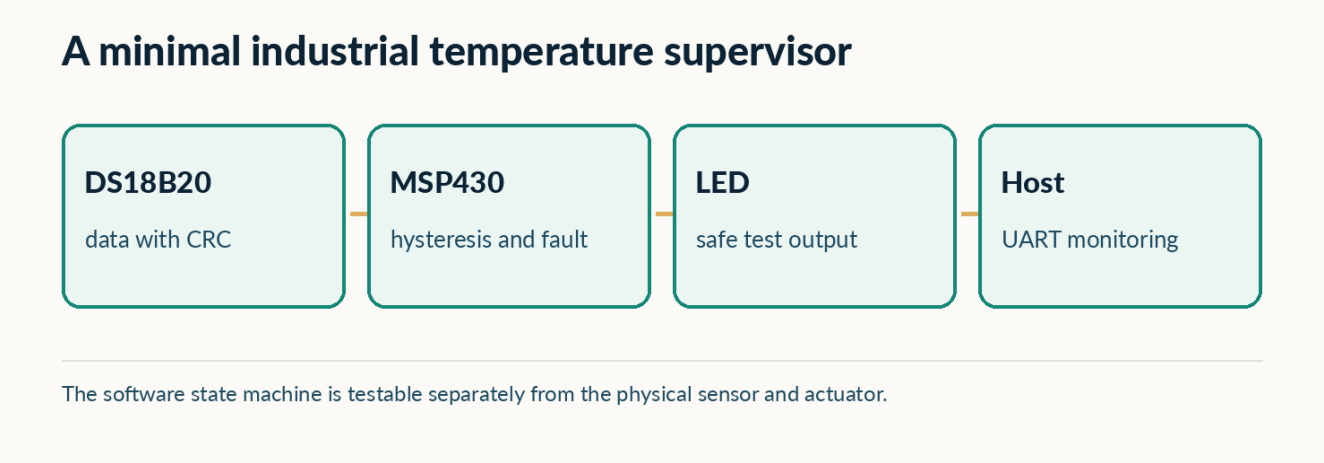


Figure 9.1 — Conceptual illustration for Chapter 9; not a verified wiring diagram.

Field laboratory 09: The Electrical Grammar of One Wire

Bus release, timing slots, and sensor presence

Industrial or historical context

The DS18B20 1-Wire protocol is exceptionally educational because the wire is not simply driven to zero or one by one participant. The idle line is high because a pull-up supplies current. The master and devices assert low, then release the bus. A master that actively drives high during a device response can create electrical contention. The simplest correct mental model is therefore not “send one and zero”; it is “pull low and release”, followed by carefully timed sampling. That is exactly the kind of low-level behavior the PDP-11 era taught engineers to examine at hardware interfaces.

Model and technical reasoning

The device data sheet defines reset, presence, write-zero, write-one, and read time slots. A reset begins with a master low interval of at least 480 µs. After release, a sensor may assert a presence pulse in a defined response window. A valid software delay count alone is insufficient evidence: rising edges are influenced by the pull-up resistor, bus capacitance, output-direction switching, and compiler-generated code. At the present 1 MHz DCO stage, source-level cycle delays must be bench-characterized. Work with externally powered three-wire mode rather than parasite power initially.

Prediction before measurement

Sketch the reset and presence waveform with ownership labels. Mark the interval in which the master drives low, the interval in which nobody actively drives, and the interval in which the sensor can pull low.

Procedure and evidence gates

1. Verify the DS18B20 package orientation and a single pull-up to the same 3.3 V logic domain.
2. Confirm the candidate P1.7 net is free from active I²C devices or alternative outputs.
3. With the sensor absent, measure bus idle-high voltage and the master reset pulse.
4. Attach one externally powered sensor; measure the presence pulse without interpreting a single low reading as sufficient.
5. Independently capture write-zero, write-one, and read slots; compare against the manufacturer’s timing table.

Interpretation questions

1. Why does internal GPIO pull-up resistance often require additional review?
2. Why is 1 kHz frequency measurement not a test of a 6 µs low pulse?
3. What could produce a false presence detection?
4. Why should interrupts be bounded or excluded only in critical short slots?

Required submission: Waveform screenshots or time-stamped edges, measured reset and presence widths, power/pull-up measurements, and explicit device-present versus device-absent test outcomes.

WORKSHEET 09 / REPRODUCIBLE LAB RECORD

The Electrical Grammar of One Wire

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 09 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

The Electrical Grammar of One Wire • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: Why does internal GPIO pull-up resistance often require additional review?

_____

_____

Prompt: Why is 1 kHz frequency measurement not a test of a 6 µs low pulse?

_____

_____

Prompt: What could produce a false presence detection?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 09 / DESIGN AND REASONING

Pull-up and rising-edge experiment

Create a controlled RC approximation using a resistor and small capacitor at safe voltage. Measure the rising-edge duration using a scope or a validated high-resolution logic acquisition, then compare two pull-up resistances. Treat the RC model as an approximation; real sensors, wiring, and thresholds may differ. Do not use this substitute as certification of the genuine DS18B20 transaction.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Provide resistor values, observed edge times, the threshold definition used, and a discussion of electrical loading and measurement bandwidth.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 10 / CORE READING

Bytes become temperature: integrity, units, and meaning

Guiding question: A useful reading requires more than correct arithmetic

Key vocabulary: Scratchpad, two's complement, CRC-8, resolution, accuracy

10.1 A conversion is more than two bytes

The DS18B20 stores its current result in a scratchpad. After a completed conversion, the two least-significant scratchpad bytes contain a signed fixed-point value. At 12-bit resolution the raw unit is 1/16 °C; the remaining scratchpad bytes include alarm/configuration values, reserved fields, and an 8-bit CRC. The CRC covers the first eight scratchpad bytes and is supplied in byte nine. A successful CRC check detects many data-transfer faults, but does not certify the sensor's thermometric accuracy or its thermal coupling to the environment.

The physical reading sequence for **one confirmed DS18B20** is: reset/presence; Skip ROM `0xCC`; Convert T `0x44`; wait for conversion; reset/presence; Skip ROM `0xCC`; Read Scratchpad `0xBE`; read nine bytes; verify CRC; decode temperature. If multiple 1-Wire devices are attached, Skip ROM is not the correct general addressing strategy for a read. Use ROM identification and device selection.

10.2 Decode signed fixed point without hiding arithmetic

Suppose bytes 0 and 1 are `0x91` and `0x01`. The raw little-endian word is `0x0191`, or 401 decimal. It represents 401/16 = 25.0625 °C, equivalent to 77.1125 °F. These are **constructed teaching bytes**; the book does not claim they came from a sensor. For a negative example, `0xFF5E` interpreted as a signed two's-complement word is -162, giving -10.125 °C. The source file `code/msp430/06_ds18b20_decode.c` implements CRC and signed decoding separately from physical bus transport.

CODE EXAMPLE · in-text excerpt

```
/* Only after a successful nine-byte scratchpad read: */
int16_t raw_x16;
int status = ds18b20_decode_x16(scratchpad, &raw_x16);
if (status == 0) {
    /* Keep the integer raw value in the target telemetry. */
    /* Celsius = raw_x16 / 16.0, preferably on host. */
} else {
    /* Report invalid reading; do not publish stale data as current. */
}
```

On the host, floating-point conversion can be used for display and plots. On the MSP430, a signed 16-bit or 32-bit fixed-point representation is often simpler. Beware of ordinary C integer division: `401 / 16` gives 25, losing the fractional component. Also avoid treating resolution as accuracy. The nominal 0.0625 °C step of 12-bit representation does not imply that the real temperature is known to that accuracy. Analog Devices specifies a typical marketed accuracy bound of ±0.5 °C across a defined central range; consult the complete electrical and operating conditions of the datasheet for actual application limits.

10.3 CRC as a protocol laboratory

The code `code/python/ds18b20_math.py` provides a host algorithm for the reflected Dallas CRC-8 polynomial, and the companion MSP430 C source uses the same bitwise logic. Start by giving learners an eight-byte synthetic message and a computed check byte. Flip one bit and observe that the CRC no longer matches. Then discuss the distinction between **correctly received bytes**, **physically correct measurement**, and **safely handled failure**.

The DS18B20 can report the power-on 85 °C register value before a completed conversion. The controller should not accept a number merely because it looks like a valid temperature. Its validity contract should require a completed conversion, successful transport, CRC integrity, and a sensible acquisition status. A CRC-valid frame can still be old or associated with the wrong sensor.

CAP gate. Supply all nine raw bytes from a real conversion, power mode, firmware version, timestamp or sample index, CRC result, signed integer decode, Celsius and Fahrenheit, and an independent approximate temperature check. Report unsuccessful transactions rather than silently discarding them.

Decode a negative temperature without magic

The DS18B20's two low scratchpad bytes form a signed sixteen-bit integer representing sixteenths of a degree Celsius at 12-bit resolution. For example, the synthetic bytes 0x5E 0xFF make 0xFF5E. Interpreted as an unsigned integer this is 65374; interpreted as signed two's complement it is −162. Dividing by 16 gives −10.125 °C.

This is an ideal boundary between embedded C and Python. The MCU should validate the bytes and publish a signed raw value with a validity flag. Python can present the temperature, graph a series, and perform further calculations. Neither language makes a reading accurate simply because it prints more digits.

Separate four claims: transmission integrity (CRC), computation correctness (fixed-point arithmetic), conversion completion (the device did finish taking a new reading), and measurement quality (the sensor accurately represents the physical object under test). Each claim requires a different type of evidence.

Worked complete sensor transaction: commands, waiting, and validity

A transaction must represent the *whole* physical operation. After an idle-high check, issue a reset and look for a genuine presence pulse. With exactly one known sensor on the bus, the Skip ROM command (0xCC) can be used, followed by Convert T (0x44). The simplified candidate waits the maximum 750 ms for a 12-bit conversion with the sensor externally powered. Reset again, select the one sensor, issue Read Scratchpad (0xBE), and collect all nine bytes. Check the CRC of bytes zero through seven against byte eight and then decode the signed raw temperature value.

This sequence deliberately makes the limitations visible. A successful CRC does not prove that the convert command was actually completed, because old or power-up scratchpad data can be returned. A result of 85 °C can be a legitimate physical measurement or the DS18B20's initial register representation; the software must establish that a new conversion was commanded and allowed to finish. Missing presence, CRC failure, unexpected sensor count, and stale data are separate fault categories that deserve distinct status codes and records.

The companion `10_ds18b20_transaction_CANDIDATE.c` is deliberately blocking and limited to one externally powered sensor. Its delay assumes an appropriate nominal 1 MHz MCU clock, and its transport functions are imported from the unvalidated 1-Wire module. A real product would introduce a nonblocking schedule and a sensor identity check. The teaching version offers an inspectable transaction before layering on an event-driven application.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

10.1 Decode the synthetic raw DS18B20 words 0x01D0 and 0xFF5E as signed Celsius values and then convert them to Fahrenheit.

10.2 Why must bytes 0–7 pass CRC comparison with byte 8 before reporting a scratchpad value?

10.3 Explain how a valid CRC and a default 85°C representation can coexist with an invalid physical inference.

10.4 Separate resolution, repeatability, absolute accuracy, and thermal response time.

10.5 Design a comparison test using an independent thermometer and an intentionally changed immersion depth.

10.6 What distinguishes a raw integer measurement from a formatted floating-point temperature string?

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/06_ds18b20_decode.c

```c
/* HOST-TESTED CRC and resolution-aware DS18B20 decoding.
 * Does NOT implement transport, conversion scheduling, or sensor accuracy.
 * Caller must validate fresh conversion and power/temperature conditions.
 */
#include "06_ds18b20_decode.h"
uint8_t onewire_crc8(const uint8_t *bytes,uint8_t len)
{
    uint8_t crc=0,i,j;
    if (!bytes && len) return 0;
    for(i=0;i<len;++i) {
        uint8_t value=bytes[i];
        for(j=0;j<8;++j){
            uint8_t mix=(uint8_t)((crc^value)&1u);
            crc >>= 1;
            if(mix) crc^=0x8Cu;
            value >>= 1;
        }
    }
    return crc;
}
int ds18b20_decode_x16(const uint8_t scratchpad[9],int16_t *out_x16)
{
    uint16_t raw,mask;
    unsigned int resolution;
    if (!scratchpad || !out_x16) return -1;
    if (onewire_crc8(scratchpad,8)!=scratchpad[8]) return -2;
    resolution=9u+((scratchpad[4]>>5)&3u);
    mask=(uint16_t)(0xFFFFu << (12u-resolution));
    raw=((uint16_t)scratchpad[0]|((uint16_t)scratchpad[1]<<8))&mask;
    if (raw&0x8000u) {
        uint16_t magnitude=(uint16_t)(~raw+1u);
        *out_x16=(int16_t)(-(int32_t)magnitude);
    } else *out_x16=(int16_t)raw;
    return 0;
}
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

LISTING · msp430/10_ds18b20_transaction_CANDIDATE.c

```c
/* DS18B20 single-sensor transaction example, MSP430G2553 (CANDIDATE).
 * NOT verified on MSP430 target. Use only with externally powered DS18B20,
 * one confirmed device, properly configured 3.3 V data pull-up,
 * 1 MHz clock, and scope-confirmed 1-Wire timing from module 09.
 * 1-Wire and UART code are intentionally separate modules.
 */
#include <msp430.h>
#include <stdint.h>
#include "06_ds18b20_decode.h"

/* Implemented in 09_onewire_transport_CANDIDATE.c */
void ow_init(void);
unsigned int ow_reset(void);
void ow_write_byte(uint8_t);
uint8_t ow_read_byte(void);

/* Returns: 0 valid conversion, -1 no presence, -2 CRC failure,
 * -3 null result pointer. It does not establish absolute accuracy.
```

```
 * Blocking conversion wait is appropriate only as a teaching baseline.
 * Never call from an interrupt handler or a timing-critical control loop.
 */
int ds18b20_read_single_x16(int16_t *result)
{
    uint8_t scratchpad[9], i;
    if (!result) return -3;
    if (!ow_reset()) return -1;
    ow_write_byte(0xCC);              /* Skip ROM: ONE device only */
    ow_write_byte(0x44);              /* Convert T */
    __delay_cycles(750000UL);         /* Up to 750 ms, nominal 1 MHz */
    if (!ow_reset()) return -1;
    ow_write_byte(0xCC);
    ow_write_byte(0xBE);              /* Read Scratchpad */
    for (i=0; i<9; ++i) scratchpad[i]=ow_read_byte();
    return ds18b20_decode_x16(scratchpad, result);
}
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

Field laboratory 10: Nine Bytes Become a Temperature

1-Wire commands, fixed-point decoding, and CRC

Industrial or historical context

Once a 1-Wire line can transmit bits reliably, the protocol can finally become a sensor instrument. The DS18B20 provides a unique ROM identity and a nine-byte scratchpad. Its first two scratchpad bytes represent signed temperature in units of 1/16 °C at 12-bit resolution. A correct byte value is not necessarily a current conversion, and an accurate display cannot be inferred from the number of fractional bits. This lab should separate the transport problem, the integrity problem, the arithmetic problem, and the actual physical temperature problem.

Model and technical reasoning

For one verified device, send reset/presence, Skip ROM (0xCC), Convert T (0x44), wait until conversion is complete, reset again, Skip ROM, Read Scratchpad (0xBE), then obtain all nine bytes. Calculate CRC over bytes 0..7 and compare with byte 8 before decoding. With raw words 0x01D0 and 0xFF5E, the mathematical values are +29.000 °C and -10.125 °C respectively; these are synthetic examples, not collected observations. A default power-on 85 °C pattern is a reason to check conversion completion rather than automatically publish.

Prediction before measurement

Compute Celsius and Fahrenheit for raw values +464 and -162 (sixteenths of a degree). Predict how a flipped scratchpad bit affects CRC acceptance.

Procedure and evidence gates

1. Test the supplied Python CRC and temperature-decoding code with known synthetic cases.
2. Record a real device ROM where possible; do not assume an arbitrary set of bytes belongs to a sensor.
3. Use the validated transport routines to run the full conversion/read sequence for one device.
4. Save raw nine-byte scratchpad transactions, CRC status, power mode, conversion settings, and timing.
5. Compare measured temperature with an independent thermometer and describe sensor placement and thermal lag.

Interpretation questions

1. Does a valid CRC demonstrate temperature accuracy?
2. What does 0.0625 °C resolution mean, and what does it not mean?
3. Why must negative values be decoded as signed 16-bit?
4. Why is the maximum 12-bit conversion time relevant to scheduling?

Required submission: A nine-byte raw transaction, CRC calculation, decoded units, conversion-completion evidence, and an independent physical comparison with limitations.

WORKSHEET 10 / REPRODUCIBLE LAB RECORD

Nine Bytes Become a Temperature

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 10 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

Nine Bytes Become a Temperature • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: Does a valid CRC demonstrate temperature accuracy?

_____

_____

Prompt: What does 0.0625 °C resolution mean, and what does it not mean?

_____

_____

Prompt: Why must negative values be decoded as signed 16-bit?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 10 / DESIGN AND REASONING

Thermal lag matters even with correct bytes

Immerse a verified sensor package at a safe, modest temperature step while keeping a second thermometer nearby. Log the time at which the physical environment changes and subsequent sensor readings. Compare two placements such as in air versus shallow water, controlling other conditions. Do not make unsupported claims about the water's uniform temperature or the sensor's absolute calibration.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Publish raw timestamped temperature data, CRC validity, a simple response-time estimate, and a note on how geometry changes the measured process.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 11 / CORE READING

Industrial control, hysteresis, and failure behavior

Guiding question: Control must define what happens when information fails

Key vocabulary: State machine, hysteresis, stale sample, safe output, fault

11.1 A controller needs memory

A naive thermostat uses one threshold: if temperature is above 29 °C, activate cooling, otherwise turn cooling off. A noisy sensor close to 29 °C can cause repeated rapid on/off decisions, a problem called chatter. Hysteresis gives the controller a memory of its previous state. In our educational example cooling becomes requested when temperature rises to 30 °C and remains requested until it falls to 28 °C. Between those values, the previous state is retained.

CODE EXAMPLE · in-text excerpt

```
               temperature rising
         28 C              30 C
OFF ------------------------------> ON
     remain in previous state
ON  <------------------------------ OFF
               temperature falling
```

The drawing describes transition thresholds, not an assertion that a real fan, heater, or thermal plant will achieve them. Without a physical actuator connected, this is an *alarm/control decision demonstrator*. To call it closed-loop control, the commanded output must measurably affect the controlled process, and the sensor must observe that effect. A simulated output LED is a useful intermediate step; it is not a substitute for actuation.

11.2 Separate the control law from the pin driver

The included `code/msp430/05_thermostat_logic.h` is deliberately device-independent, so it can be studied and tested without energizing equipment. It takes three inputs: the prior state, signed temperature in 1/16 °C, and a validity flag. Its outputs are OFF, ON, or FAULT.

CODE EXAMPLE · in-text excerpt

```
control_state_t next = control_step(previous_state, temp_x16, sensor_valid);
/* A separate, reviewed driver maps next state to a real output. */
```

When the sensor is invalid, the example returns `CONTROL_FAULT`; the actuator policy must define what that means. For a classroom LED, fail-to-off is a reasonable demonstration. For a real freezer, incubator, furnace, or fluid process, arbitrarily switching off could be dangerous. Safety policy belongs to the physical application and may require independent hardwired protections.

The pure decision function is testable with data such as 27 °C, 29 °C, 31 °C, and an invalid sensor. Students can predict the state sequence before running the test. A good test covers the threshold itself, the band between thresholds, a recovery from FAULT, and the case where a sensor returns an old but plausible value. Recovery policy should not be accidental; our sample chooses OFF if a valid temperature returns inside the band after a fault.

11.3 Scheduling a process supervisor

One feasible architecture uses a periodic timer flag to schedule sensor work, a bounded state machine to initiate and complete conversions, and a low-priority output path to transmit telemetry. A 750-ms conversion at 12-bit resolution should not be implemented as a single interrupt handler that busy-waits for 750 ms. The controller can issue

conversion, return to other work, and collect data after completion. However, the 1-Wire timing-critical slots do require carefully bounded microsecond-scale operations.

CODE EXAMPLE · in-text excerpt

```
SAMPLE_IDLE -> START_CONVERSION -> WAIT_CONVERSION
            -> READ_SCRATCHPAD -> VALIDATE_CRC
            -> UPDATE_CONTROLLER -> REPORT -> SAMPLE_IDLE
                 | any error -> FAULT -> REPORT
```

This is the same architectural separation that real-time industrial systems demand: acquisition, decision, actuation, and reporting have different timing and failure requirements. On a PDP-11 with RSX-11M, the responsibilities might be distributed among driver interrupt service, prioritized tasks, and data-logging processes. On the MSP430 they can be functions in a compact event-driven loop. Compare the responsibilities, not the software scale.

11.4 A verification matrix, not a show-and-tell

Test at least four independent properties. **Sensor integrity:** disconnect DQ or corrupt a CRC and verify FAULT. **Control correctness:** sweep temperatures upward and downward across both thresholds, verifying transitions. **Timing:** measure acquisition interval and any output transition delay. **Independence:** disconnect the PC display during an active experiment; the local MCU logic should continue its documented behavior. If it does not, the architecture has unintentionally moved control into the observation path.

CAP gate. Record a state-transition table including invalid data and recovery, wiring, raw values, state outputs, and one failure-injection test. Conclude only what the test establishes.

A fail-safe policy is a separate specification

A threshold controller with hysteresis remembers a state. A cooler is commanded ON at or above 30 °C and OFF at or below 28 °C. Inside that interval, the prior valid command persists. This prevents needless rapid switching when temperature fluctuates near a single trip point.

Now suppose the sensor disappears. Is “last command forever” acceptable? Usually not without a designed and justified policy. For a classroom indicator, the safe state can be LED OFF plus a separate fault LED ON. A real industrial system may require a different protective outcome, depending on what the actuator does and which hazards dominate. The safe state must be defined by the process requirements and then implemented and tested.

The algorithm can be tested on a normal PC using integer temperature data and synthetic faults. That is an economical method for verifying transitions before any wire is connected. The tests are important evidence about the algorithm, but not about sensor wiring, electrical noise, or the response of a physical fan.

A supervisory system is more than a thermostat

Consider a simulated industrial supervisor that reads temperature, chooses a cooling-demand state, announces it over UART, and uses LEDs to expose faults. A cooling request becomes ON at 30 °C or above and OFF at 28 °C or below. Between those boundaries, the last valid state persists. If the sensor is invalid, the teaching policy requests the FAULT state, with the active output disabled. Upon the next valid sample inside the hysteresis band, this implementation returns to OFF rather than silently restoring the pre-fault ON command.

That policy is suitable only for a harmless classroom LED. Whether an actual industrial plant should open a valve, close a valve, stop a pump, or run an emergency fan upon sensor failure cannot be decided by a generic C function. It depends on hazard analysis, actuator polarity, independent interlocks, and the cost of both false positive and false negative action. Students should write a process-specific safe-state rationale before proposing anything stronger than an LED.

The included `11_supervisor_demo_HOST.c` runs a sequence of eight synthetic measurements and prints a CSV-compatible state trace. Its 1/16-degree integer arithmetic is the same representation used by the DS18B20 decoding code. The trace establishes the decision function's behavior on those cases but gives no evidence about any attached sensor or output hardware. Challenge the student to add a test case for exactly 28 °C, exactly 30 °C, a loss of sensor validity, and recovery with a new valid sample. Verify them on the host before porting to a target.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

11.1 Trace OFF/ON transitions for 27.5, 29.0, 30.0, 29.0, 28.0°C using 30°C ON and 28°C OFF thresholds.

11.2 Construct a transition table for exactly 28°C and 30°C in each possible prior state.

11.3 Define a fault policy for no sensor, CRC failure, and a stale sample. What evidence would prove each branch executes?

11.4 Why should a logical cooling request not be equated with an energized motor?

11.5 Explain why test results on a PC can validate a state machine without validating wiring or thermal behavior.

11.6 Specify one boundary condition that unit tests routinely overlook.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · msp430/05_thermostat_logic.h

```
/* Platform-independent demonstrator state machine. No mains/load driver.
 * Fractional Celsius value represented as signed 1/16 degree integer.
 */
#ifndef THERMOSTAT_LOGIC_H
#define THERMOSTAT_LOGIC_H
#include <stdint.h>
typedef enum { CONTROL_OFF=0, CONTROL_ON=1, CONTROL_FAULT=2 } control_state_t;
static inline control_state_t control_step(control_state_t old, int16_t temp_x16,
                                           uint8_t sensor_valid) {
    if (!sensor_valid) return CONTROL_FAULT;
    if (temp_x16 >= 30 * 16) return CONTROL_ON;
    if (temp_x16 <= 28 * 16) return CONTROL_OFF;
    if (old == CONTROL_ON) return CONTROL_ON;
    return CONTROL_OFF; /* fail-safe recovery in hysteresis band */
}
#endif
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

LISTING · msp430/11_supervisor_demo_HOST.c

```
/* Host-executable controller demonstration. No MCU I/O; no real loads.
 * Compile: cc -std=c11 -Wall -Wextra -Werror -Icode/msp430 \
 *    code/msp430/11_supervisor_demo_HOST.c -o supervisor_host
 */
#include <stdio.h>
#include <stdint.h>
#include "05_thermostat_logic.h"
int main(void)
{
    const struct {int16_t x16; uint8_t valid;} sequence[] = {
        {27*16,1},{29*16,1},{30*16,1},{29*16,1},
        {29*16,0},{29*16,1},{31*16,1},{28*16,1}
    };
```

```
14      control_state_t state=CONTROL_OFF;
15      size_t i;
16      puts("sample,temp_C,valid,state");
17      for (i=0; i<sizeof(sequence)/sizeof(sequence[0]); ++i) {
18          state=control_step(state,sequence[i].x16,sequence[i].valid);
19          printf("%u,%.4f,%u,%d\n",(unsigned)i,
20                 sequence[i].x16/16.0,(unsigned)sequence[i].valid,(int)state);
21      }
22      return 0;
23  }
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

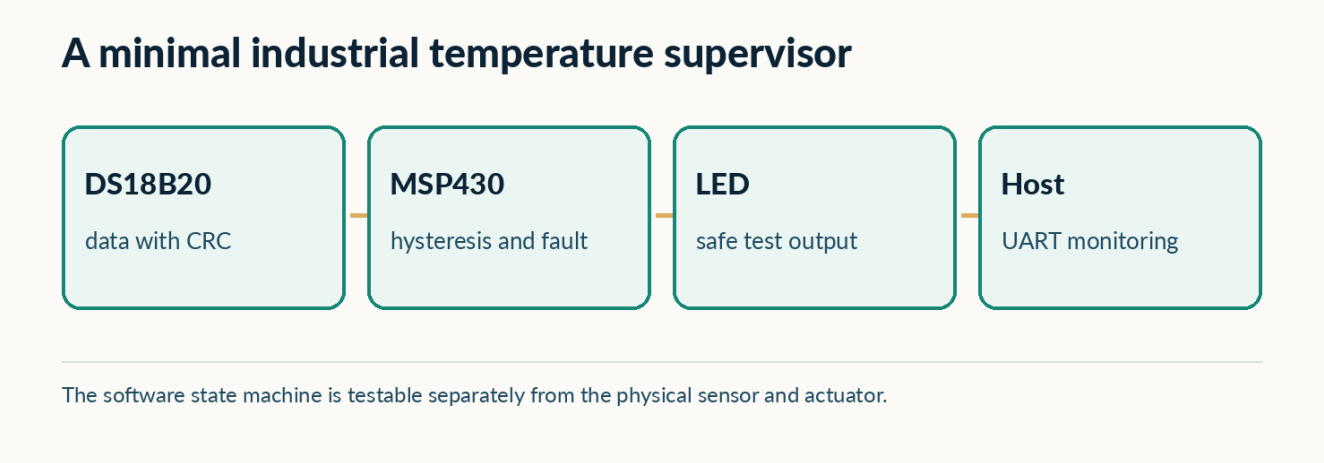


Figure 11.1 — Conceptual illustration for Chapter 11; not a verified wiring diagram.

Field laboratory 11: A Temperature Supervisor That Fails Safely

Hysteresis, state machines, and fault injection

Industrial or historical context

The capstone controller uses the smallest possible control law that still demonstrates memory and state. A cooling-request indicator turns ON at or above 30 °C and turns OFF at or below 28 °C. Between these thresholds it remembers its previous legitimate state. If the input is missing, CRC-invalid, out of date, or otherwise inadmissible, the program enters FAULT and sets a separately defined safe output state. This is an educational output indicator, not a qualified safety controller and not a mains-connected thermostat.

Model and technical reasoning

The controller receives temperature as signed integer sixteenths of a degree, matching the DS18B20 raw scale. `control_step` produces OFF, ON, or FAULT; a separate driver translates state into an output latch. That separation is valuable because the algorithm can be tested on a host PC without invoking an MSP430 header, simulator, or sensor. The software can be wrong even when the sensor bytes are correct, so exercise threshold boundaries and transition history explicitly. A failed sensor read should never be silently substituted by the last good value without showing the age and a defined policy.

Prediction before measurement

Starting in OFF, run this sequence: 27.5, 29.0, 30.0, 29.0, 28.0 °C. Predict the resulting state after each sample. Then inject one CRC-invalid sample and predict FAULT.

Procedure and evidence gates

1. Compile and run the platform-independent host tests of the state machine.
2. Construct a transition table with values just below, equal to, and just above both thresholds.
3. Inject invalid CRC and stale data as distinct simulated faults; document how the input validity flag is obtained.
4. Only after host tests pass, integrate with verified sensor acquisition and an LED output.
5. Demonstrate that disconnecting the host UART has no effect on the local state progression.

Interpretation questions

1. What is the benefit of hysteresis over a single threshold?
2. Why must the fault output action be defined independently from control state names?
3. How can the host PC prove the algorithm but not prove the entire controller?
4. What additional evidence is needed before using a real fan or mechanical actuator?

Required submission: Complete transition table, passing host test transcript, one injected fault, a local-output bench demonstration, and a safety limitation statement.

WORKSHEET 11 / REPRODUCIBLE LAB RECORD

A Temperature Supervisor That Fails Safely

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 11 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

A Temperature Supervisor That Fails Safely • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: What is the benefit of hysteresis over a single threshold?

_____

_____

Prompt: Why must the fault output action be defined independently from control state names?

_____

_____

Prompt: How can the host PC prove the algorithm but not prove the entire controller?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 11 / DESIGN AND REASONING

Inject failures before adding a real actuator

Run a platform-independent supervisor with injected CRC faults, timeout faults, readings exactly at 28 and 30 °C, and a rapidly oscillating series near each threshold. Confirm that the state-machine policy is explicit and stable. Do not interpret an LED as proof that a motor driver or fan would be electrically safe.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Deliver a transition matrix with every input, previous state, new state, output request, and reason, plus a separate statement defining the required physical fault output.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

CHAPTER 12 / CORE READING

Publish an experiment that another engineer can reproduce

Guiding question: Evidence turns an experiment into shareable engineering

Key vocabulary: Traceability, raw data, uncertainty, reproducibility, provenance

12.1 Evidence is a design product

Industrial computing did not become credible solely because computers were faster than relays. It became credible through documented interfaces, repeatable procedures, diagnosis, and control of configuration. The same habits can be taught using a modern editor, a small microcontroller, and an inexpensive measuring instrument. A scientific report should state the intended behavior, the actual build, the observed evidence, the instrument's limits, and what would falsify the conclusion.

At minimum, preserve: board model and revision, electrical supply, sensor identity and power mode, wiring drawing, compiler/toolchain version, source hash or version tag, programmed image, measurement firmware/configuration, raw measurements, host-analysis script version, and a photograph or schematic of the actual experimental arrangement. A teacher should never substitute a synthetic expected measurement for a missing physical observation; mark the experiment incomplete.

12.2 Python as the laboratory notebook engine

The companion `code/python/cap_analyze.py` reads an honest frequency CSV, calculates basic descriptive statistics and difference from nominal, and publishes a small Markdown report. The `worksheets/frequency_capture_template.csv` provides blank fields rather than plausible pre-filled numbers. Every field should be treated as a measurement contract: units and instrument state must be present, and missing observations must remain missing.

A reporting script cannot independently calibrate the Pico or the MSP430. It can calculate `mean`, `min`, `max`, `range`, and standard deviation, but the meaning of these statistics depends on the sampling protocol. Five measurements that share the same incorrect reference clock can be closely clustered and all wrong. Report repeatability and evidence of calibration separately.

12.3 An uncertainty budget in plain language

For the 1-kHz test, plausible error contributors include the MSP430 DCO frequency, Pico reference crystal, gate timing, and integer count quantization. A rectangular one-second gate with count N has an idealized one-count granularity of about one hertz; the actual system may include other timing errors. If the measurement system later uses an external GPS 1 PPS signal to characterize its oscillator, confirm GNSS lock, missed pulses, event selection, and the counter implementation. An average of several intervals reduces some random variation but cannot by itself eliminate constant bias or misidentified timing edges.

The objective of a classroom uncertainty budget is not to manufacture a precision claim. It is to teach the student to ask: what was compared to what, for how long, under which conditions, and how could the conclusion be wrong?

12.4 A defensible final demonstration

Ask students to present a ten-minute evidence-based demonstration. Their controller should read temperature or report an explicit fault, calculate a control state, emit truthful telemetry, and preserve all raw sensor bytes. The Pico should provide at least one independent timing observation, not necessarily continuously. The final report should compare measured behavior to the stated requirements and distinguish any unfinished hardware validation.

A passing project need not have a beautiful dashboard or advanced network infrastructure. The most important product is *explainable behavior* that remains useful without the PC. The PDP-11's industrial heritage is honored when students understand their sensor, timing, decisions, and failures—not when they merely reproduce a vintage machine in miniature.

The strongest report contains an anomaly

A laboratory report is not merely a success announcement. If the 1 kHz meter reports 998 Hz during one trial, record that value and the test conditions. Repeat the measurement, check the gate configuration, and change one controllable variable. A result outside expectation is potentially more informative than a perfect-looking output, especially when it exposes a hidden assumption about the oscillator, loading, or gate timing.

Preserve raw bytes and counts, not just interpreted temperatures and finished charts. Save the source version, compiler invocation, target part, wiring, meter settings, and date. This makes the work recoverable after a reboot, a device replacement, or a long interval between lessons.

The student’s final assertion should be no stronger than the evidence. “This board produced five readings near 1 kHz with the stated settings” is defensible after those captures. “This chip has an absolute clock accuracy of one part per million” would require a separate reference and uncertainty analysis.

A repeatable experiment includes a software bill of materials

A source archive without versions or run instructions is difficult to reuse, particularly after a reboot or long gap between sessions. Every published experiment should include a short manifest: target device, PCB revision, toolchain and linker configuration, source commit or archive checksum, programming procedure, I/O wiring, instrument firmware, measurement settings, and raw output. Include an environment note stating what was not available or tested.

For the software validation exercise, run `tests/run_host_tests.sh` on a computer with Python and a C compiler. The script tests the CSV summarizer, scratchpad CRC and signed decoding, the hysteresis state machine, and a human-readable host demonstration. A passing host test validates those functions under the tested inputs, not a complete microcontroller program. An appropriate publication includes both the passing log and the separate target-compilation and physical test records when those are eventually obtained.

For a publication-ready figure, label each axis with units, distinguish measured from computed values, show actual sample count, identify the independent timing reference if one exists, and discuss the limits imposed by count quantization and timebase drift. A neat graph that omits its reference clock is less informative than a simple table that records every raw count and timestamp.

Review and applied problems

These problems assess interpretation and engineering judgment, not vocabulary recall alone. Work before reading the instructor key.

12.1 Write a defensible claim based on five readings around 1000 Hz; write a stronger claim that would require a calibrated reference.

12.2 Calculate mean and range for synthetic frequency values 999, 1000, 998, 1001, 1000 Hz.

12.3 List metadata necessary for independent replication after a PC reboot six months later.

12.4 Explain how selective deletion of outlier measurements can bias the interpretation.

12.5 Distinguish an observed measurement, predicted model value, and formatted dashboard summary.

12.6 Develop a one-page evidence checklist for the final temperature-supervisor demonstration.

End-of-chapter challenge: Write a one-paragraph explanation that links one historical architectural choice with one measurable effect on your MSP430 bench.

Worked source listing

Examine the source before changing it. Explain each register, constant, and externally observable effect; compile for the specified target and save all diagnostics. Code included here is educational candidate material, not a bench certification.

LISTING · python/cap_analyze.py

```
  1   """CAP analysis for observed frequency measurements; no acquisition or calibration."""
  2   import csv,math,statistics,sys
  3   from pathlib import Path
  4
  5   def summarize(path, target=1000.0):
  6       if not math.isfinite(target) or target<=0: raise ValueError("Nominal target must be positive and finite")
  7       values=[]
  8       with Path(path).open(newline="",encoding="utf-8-sig") as fh:
  9           table=csv.DictReader(fh)
 10           if not table.fieldnames or "reading_hz" not in table.fieldnames:
 11               raise ValueError("CSV requires a reading_hz column")
 12           for lineno,row in enumerate(table,2):
 13               if None in row: raise ValueError(f"CSV row {lineno}: too many fields")
 14               s=(row.get("reading_hz") or "").strip()
 15               if not s: raise ValueError(f"CSV row {lineno}: blank measurement")
 16               try: x=float(s)
 17               except ValueError as e: raise ValueError(f"CSV row {lineno}: invalid number") from e
 18               if not math.isfinite(x) or x<0: raise ValueError(f"CSV row {lineno}: nonfinite or negative value")
 19               values.append(x)
 20       if not values: raise ValueError("No actual measurements recorded")
 21       mean=statistics.mean(values)
 22       return {"n":len(values),"mean_hz":mean,"min_hz":min(values),"max_hz":max(values),
 23               "range_hz":max(values)-min(values),"sample_stdev_hz":statistics.stdev(values) if len(values)>1 else
0.0,
 24               "difference_from_nominal_hz":mean-target,"difference_pct":100*(mean-target)/target}
 25
 26   if __name__=='__main__':
 27       if len(sys.argv) not in (2,3): raise SystemExit("Usage: python cap_analyze.py measurements.csv
[nominal_hz]")
 28       result=summarize(sys.argv[1],float(sys.argv[2]) if len(sys.argv)==3 else 1000.0)
 29       for k,v in result.items(): print(f"{k}: {v:.5f}" if isinstance(v,float) else f"{k}: {v}")
```

Code status: target source candidate unless independently compiled, flashed, and measured. See validation notes.

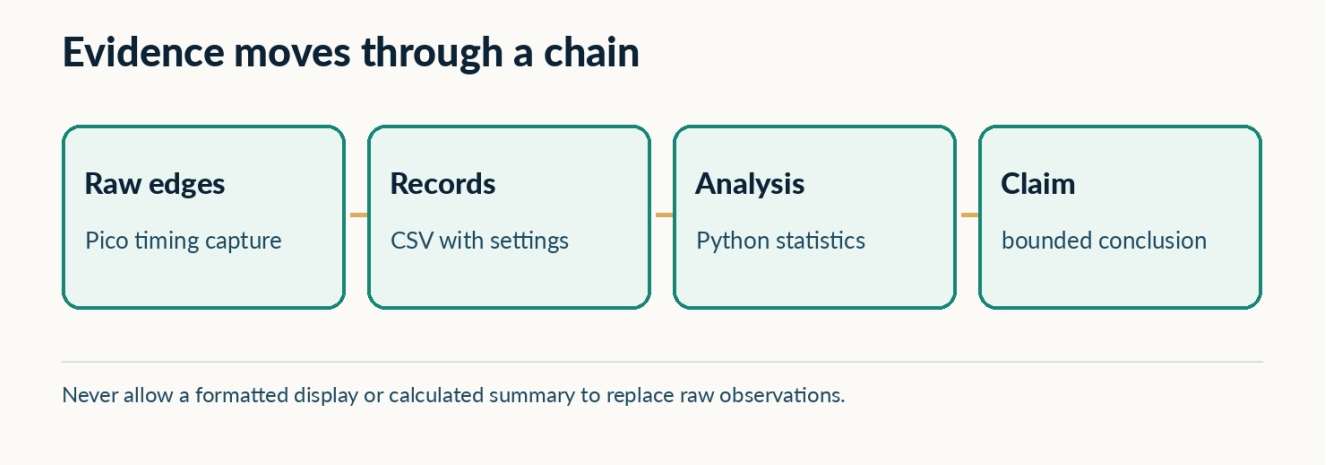


Figure 12.1 — Conceptual illustration for Chapter 12; not a verified wiring diagram.

Field laboratory 12: The Engineering Notebook as a Public Artifact

Capture, Analyze, Publish and independent replication

Industrial or historical context

A working demonstration is a beginning, not a finished engineering result. The temptation to report only clean numbers is particularly strong in classrooms, where failures consume time and a beautiful dashboard can look more authoritative than a handwritten record. The CAP method counters that temptation by preserving raw observations, recording the actual instrument configuration, and naming limitations that cannot be removed by formatting. In industrial automation, these habits are close relatives of calibration control, change management, commissioning records, and maintenance logs.

Model and technical reasoning

The report should identify the actual hardware and software versions, a question, an a priori prediction, a clear schematic or annotated wiring photo, raw measurements, calculations, uncertainty discussion, faults encountered, and a falsifiable conclusion. A sample frequency with many decimal places may be repeatable without being accurate. A scratchpad with a correct CRC may be internally consistent while reporting the temperature of a badly mounted sensor rather than the process. The report should make those distinctions visible.

Prediction before measurement

Write one claim you expect to be able to defend about the 1 kHz waveform and another claim you cannot defend without a calibrated reference. Put a specific test against each.

Procedure and evidence gates

1. Inventory the source, firmware, board, sensor, Pico, and host tool versions.
2. Choose one real dataset and preserve the original CSV or raw terminal capture read-only.
3. Run the Python analyzer, check its output by hand, and add the measurement conditions.
4. Include the strongest failure or anomaly rather than silently deleting it.
5. Ask an independent reader to reproduce the calculation and identify one alternative explanation.

Interpretation questions

1. How is a measurement different from a calculated expectation?
2. What is missing from a report containing only a chart?
3. What is the minimum proof of a successful 1-Wire transaction?
4. Which revision-control facts should survive a reboot months later?

Required submission: A complete CAP paper, raw data appendix, source hashes, repeatability and uncertainty explanation, feedback from an independent reviewer, and clear status labels.

WORKSHEET 12 / REPRODUCIBLE LAB RECORD

The Engineering Notebook as a Public Artifact

Name / team: _____ Date: _____

Target board and revision: _____ Instrument: _____

Firmware or simulation revision: _____ Clock / settings: _____

Question, prediction, and test conditions

_____

_____

_____

Actual observations — never substitute predicted numbers

Trial / state	Observed result	Units / settings	Note / anomaly
1			
2			
3			
4			
5			

Analysis and limitations

Calculated comparison:

_____

What changed, and why?

_____

What can these observations NOT establish?

_____

Next controlled experiment:

_____

FIELD RECORD 12 / ANALYSIS, FAULTS, AND REPLICATION

The evidence behind the result

The Engineering Notebook as a Public Artifact • Complete only after collecting real data or recording a documented failed attempt.

Equipment and exact reproducibility information

Board / part / revision: _____

Firmware source path, version, and build command: _____

Instrument, reference and live configuration: _____

Wiring diagram, power, clock and signal conditions: _____

Explain your observation

Prompt: How is a measurement different from a calculated expectation?

_____

_____

Prompt: What is missing from a report containing only a chart?

_____

_____

Prompt: What is the minimum proof of a successful 1-Wire transaction?

_____

_____

Failure and uncertainty log

Observed anomaly	Alternative explanation	Next discriminating test

Publication decision

Claim supported by the record: _____

Claim not yet supported: _____

Instructor / reviewer sign-off: _____ Date: _____

ADVANCED FIELD INVESTIGATION 12 / DESIGN AND REASONING

Reproduce somebody else’s result

Exchange a carefully labeled evidence package with another learner. The receiver must reconstruct the claimed frequency or temperature result without verbal assistance. The receiver should record any missing firmware, wiring, clock settings, sensor identity, or raw data. Revise the package once and show whether the second attempt succeeds.

Design worksheet

Question and independent variable:

_____

_____

Controlled conditions and the alternative hypothesis:

_____

_____

Prediction and expected order of magnitude:

_____

_____

Capture table

Trial / condition	Expected	Observed	Uncertainty / note

Deliverable

Deliver both the original and revised evidence-package manifest, the reviewer’s findings, and a specific statement of reproducibility limits.

Verification note: Record failed or absent observations explicitly. Do not replace evidence with an ideal mathematical example.

INDUSTRIAL COMPUTING / FOUNDATIONS AND PRACTICE

Part III — Teacher's and operator's companion

HISTORY • ARCHITECTURE • MEASUREMENT • CONTROL

CHAPTER 13 / CORE READING

Twelve-session teaching plan

Guiding question: The operator and instructor have different responsibilities

Key vocabulary: Readiness gate, formative assessment, operator checklist

Each session assumes 75–90 minutes after hardware setup. The pedagogical pattern is Predict (10 minutes), Explain (15), Build/observe (35), Analyze (15), Publish (10). Extend the hands-on period when students encounter genuine debugging. The outcome is a structured record, not a claim of completion regardless of evidence.

Session	Historical/architectural idea	Physical or simulated investigation	Minimum evidence
1	PDP-11 as industrial controller	Draw an acquisition/control system	Labeled interfaces and failure modes
2	PDP-11 registers and modes	Trace MACRO-11-style register loop	Register-state table
3	Architecture tradeoffs	Compare PDP-11/MSP430 syntax and encodings	Annotated instruction comparison
4	Timer vs CPU	MSP430 1-kHz square wave	Five measured gates and wiring
5	Compiler behavior	C/assembly Timer_A comparison	Listing plus paired measurements
6	Memory-mapped I/O	Read switch, drive LED	Measured logic truth table
7	Interrupts and deadlines	Timer-driven interrupt output	Timing and variation plot
8	UART and observability	Send known character and telemetry	Captured terminal bytes
9	Timed serial protocol	Verify 1-Wire slots and presence	Pin-level waveforms
10	Integrity and temperature	Read nine scratchpad bytes and check CRC	Raw bytes, CRC, C/F conversion
11	Industrial decision logic	Hysteresis and injected faults	Transition/failure matrix
12	Reproducible engineering	Capstone CAP report	Versioned evidence bundle

13.1 Three teaching levels

A newcomer should explain cause and effect with a wiring drawing and a small source slice. An intermediate student should predict numeric behavior, read a disassembly, and collect genuine measurements. An advanced student should examine ISR latency, oscillator uncertainty, code-size/encoding tradeoffs, and driver timing. All three levels should meet the same standard of honesty about the observed evidence.

13.2 Grading rubric

Allocate equal weight to (1) safe and traceable setup, (2) model and prediction, (3) real observation and analysis, and (4) reproducible publication. Award credit for a well-documented failure with a sound diagnosis. Do not award a fabricated five-reading data table. A student should be able to state which subsystem generated each claimed signal and which instrument observed it.

13.3 The instructor's stop list

Stop and investigate before applying power if the board is unpopulated, unknown, reversed, connected to competing supplies, or exposed to 5-V GPIO. Preserve MCU information flash calibration and do not use erase-all as a workaround. Do not combine stand-alone Timer_A examples into one program without resolving peripheral ownership and ISR-vector conflicts. Do not trust sensor data until transport and CRC are verified. Keep high-voltage and hazardous actuators entirely outside the introductory build.

Field-operation rubric

Safety and pin-map preflight must pass before energizing the board.

Every practical experiment includes one pre-registered prediction and a measured result.

A failed bench run remains a valid documented investigation, never an invented successful measurement.

Instructor notes are placed in the end matter, away from student worksheets.

CHAPTER 14 / CORE READING

The historical bridge: what survives fifty years?

Guiding question: Continuity of ideas, not identity of hardware

Key vocabulary: Architecture, modularity, real time, instrumentation, deployment

The PDP-11, launched in 1970, established a notably coherent programming model for small computers. Through an ecosystem of hardware modules, data acquisition equipment, MACRO-11, and real-time systems such as RSX-11M, it participated in the physical automation of industrial work. In DEC's own 1977 *Introduction to RSX-11M*, oil refineries and steel rolling mills appear as examples of monitored and controlled processes. The historical claim is not that every industrial controller was a PDP-11, but that the PDP-11 was an important, documented platform for programmable real-time control.

The MSP430G2553 is a very different machine but retains features worth comparing: a 16-bit programmer's view, addressable PC and SP, two-operand operations, memory-mapped peripherals, interrupts, and modest enough resources that machine behavior remains visible. The extra registers and compact constant generator come with addressing-mode restrictions. Jonathan Kirwan's July 18, 2003 essay, *A Perspective on the MSP-430 Instructions and Addressing Modes*, is a useful contemporary discussion of that precise tradeoff. It also reminds us that TI's `RISC` marketing label should not replace a careful understanding of instruction encoding and actual execution cost.

The Pico instrument adds a third and equally important idea. Rather than judging the MSP430 by whether a program compiles, the class measures pulses, examines timing, records error, and publishes evidence. The development hardware may be sophisticated while the deployed controller is inexpensive. That separation of development capability from deployment cost is as educationally important as the individual instruction sets.

The historical bridge can be summarized as **architecture → real-time operation → physical process → independent measurement → explainable evidence**. This is the course's durable idea. It can be extended to an STM32, an ESP32, a modern industrial PLC, or a simulated historical minicomputer without losing the fundamental engineering questions.

INDUSTRIAL COMPUTING / FOUNDATIONS AND PRACTICE

Appendices

HISTORY • ARCHITECTURE • MEASUREMENT • CONTROL

Appendix A — Compact PDP-11 and MSP430 register comparison

Concept	PDP-11	MSP430G2553
Data path	16-bit	16-bit
Visible CPU registers	R0–R7	R0–R15
General working registers	R0–R5	R4–R15
PC	R7	R0
SP	R6	R1
Status/flags	Processor Status Word	R2 Status Register
Constant generator	No dedicated counterpart	Encodings involving R2/R3
Two-operand operations	Yes	Yes
General autoincrement destination	Yes	No
Memory-mapped I/O	Yes	Yes
Target context	Minicomputer and peripheral system	On-chip microcontroller peripherals

Instruction formats and semantics are not interchangeable. The PDP-11 has broader symmetrical operand modes, while the MSP430 uses different fields for its source and destination. The MSP430's emulated instructions and constant generation should be learned from TI's CPU chapter, not inferred from one assembler's mnemonics.

Appendix B — Safe bench reference

Treat these as proposed custom-board connections, to be confirmed against the actual PCB and schematic: P1.6/TA0.1 to Pico GP3 for frequency measurement; P1.7 to DS18B20 DQ with one verified ~4.7-kΩ 3.3-V pull-up; P1.2 UART TX and P1.1 RX; P2.0 switch input; P1.0 LED output. Use debugger-side Spy-Bi-Wire from the G2ET eZ-FET, matching labeled 3.3 V, GND, RST, and TEST. Use regulated single-source target power and a common reference ground. Keep GPIO below the relevant device absolute maximum and never use 5-V logic directly.

The board documents and earlier Teacher/Operator Guide remain the controlling source for physical wiring. This book does not certify the PCB as manufactured or populated.

Appendix C — Operator worksheet: capture, analyze, publish

Prediction: What exact observable event should occur and what numeric value is expected? **Configuration:** target power, pin map, program hash, clock source, meter firmware, gate duration, acquisition settings. **Capture:** preserve raw data without rewriting, including failures. **Analyze:** show equations, observed variation, one limitation, and an alternative explanation. **Publish:** concise claim, strongest evidence, configuration needed for repetition, and next controlled test.

A reproducible lab should always contain a **status label**: historical demonstration, host-tested algorithm, candidate target code, bench-observed physical behavior, or verified calibration. This prevents a source example from evolving into an unsupported claim as the curriculum is redistributed.

Appendix D — References and provenance

1. Digital Equipment Corporation, *Introduction to RSX-11M*, Version 3.1, December 1977, especially Chapter 2, process-control and data-acquisition applications. https://bitsavers.trailing-edge.com/pdf/dec/pdp11/rsx11m_s/RSX11M_V3.1_Dec77/1A_System_Reference_Information/AA-2555C-TC_Introduction_to_RSX-11M_3.1_Dec77.pdf 2. DEC, *PDP-11 Processor Handbook* and *PDP-11 Peripherals Handbook*, multiple editions, archived at Bitsavers. https://bitsavers.org/pdf/dec/pdp11/ 3. Texas Instruments, *MSP430F2xx, MSP430G2xx Family User's Guide*, SLAU144K. https://www.ti.com/lit/ug/slau144/slau144.pdf 4. Texas Instruments, *MSP430G2553 Datasheet*. https://www.ti.com/lit/ds/symlink/msp430g2553.pdf 5. Texas Instruments, *MSP-EXP430G2ET LaunchPad Development Kit User's Guide*. https://www.ti.com/lit/ug/slau772/slau772.pdf 6. Analog Devices (Maxim Integrated), *DS18B20: Programmable Resolution 1-Wire Digital Thermometer*, Rev. 6. https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf 7. Jonathan Kirwan, *A Perspective on the MSP-430 Instructions and Addressing Modes*, embedded-systems discussion, July 18, 2003. Historic author analysis supplied by reader; treat opinions as attributed rather than definitive processor taxonomy. 8. *MSP430 Day One Teacher & Operator Guide* and *MSP430 Teacher & Operator Guide v2.0*, October 29, 2026, user-provided project references, with code audit and bench-validation caveats.

Historical accuracy note. The instruction-set similarities do not establish that the MSP430 was directly derived from the PDP-11. The industrial descriptions in this book describe documented capabilities and representative system engineering, not reconstruction of a specific customer's proprietary plant program.

Appendix E — Reference implementation inventory

• `code/pdp11/01_register_loop.mac`: conceptual MACRO-11-style loop; assemble in a confirmed PDP-11 toolchain before describing as executed. - `code/msp430/01_timer_1khz.c`: nominal Timer_A 1-kHz output; target compilation/physical validation needed. - `code/msp430/02_gpio_switch.c`: active-low button and LED; board polarity validation needed. - `code/msp430/03_interrupt_led.c`: Timer1_A interrupt demonstration; target compilation/measurement needed. - `code/msp430/04_uart_hello.c`: UART test at nominal 1 MHz and ~9600 baud; verify actual clock/baud. - `code/msp430/05_thermostat_logic.h`: pure digital supervisor function; no mains/load connection. - `code/msp430/06_ds18b20_decode.c` and `.h`: CRC and signed scratchpad decoder, separate from bus acquisition. - `code/msp430/07_onewire_transport_design.txt`: proposed physical-layer contracts and evidence requirements; **not runnable driver firmware**. - `code/msp430/09_onewire_transport_CANDIDATE.c`: experimental timing primitives for review and scope-based validation; **not bench-proven**, and must not be merged into the Timer_A frequency example without redesign. - `code/python/ds18b20_math.py`, `code/python/cap_analyze.py`: host data examples with unit tests; not instruments or clocks. - `worksheets/`: blank temperature and frequency capture forms. - `tests/test_host.py`: tests host-based arithmetic and parsing only; not target CPU, real I/O, real timing, or calibration.

APPENDIX F / INSTRUCTOR EDITION

Answer key and common misconceptions

The following responses are instructional models, not evidence of a completed physical experiment. A correct conceptual answer does not replace a board measurement or a toolchain log. The teacher may accept technically equivalent explanations that account for all observed states and units.

INSTRUCTOR LAB 01 / INSTRUCTION AND ASSESSMENT

Field laboratory 01 — Teacher notes

The oscilloscope observes the electrical event directly, subject to its own calibration and probe loading. The local controller can keep executing when the PC is absent, but that is a design requirement to be demonstrated. An emergency stop must use suitable independently engineered protective hardware; a loop in a small teaching CPU cannot be assumed to provide the necessary safety function. Modular buses and peripherals supported adaptation to varied instruments and processes.

Before sign-off: One annotated block diagram; one interface contract table; a priority/deadline table; a fault-response sketch; a signed statement of what was assumed rather than observed.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• What would an oscilloscope prove that a printed telemetry line cannot?

• Which components continue operating if the host PC powers down?

• Why is a watched software loop not the same as an emergency-stop circuit?

• What historical advantage did modular DEC interfaces offer plant engineers?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 02 / INSTRUCTION AND ASSESSMENT

Field laboratory 02 — Teacher notes

A programmer may address registers directly, but R6 and R7 have important execution roles and should not be treated as scratch storage casually. SOB reduces instruction count for a common loop pattern. Simulation proves behavior of the configured emulation and the loaded binary; it does not prove real peripheral timing or the electrical behavior of a 1970s machine.

Before sign-off: Source listing, a five-row trace table, final register dump, simulator command record, and an explanation of octal versus decimal representation.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• How do the register aliases SP and PC affect ordinary arithmetic instructions?

• Why might a programmable machine provide a single decrement-and-branch instruction?

• What is gained by using an emulator first, and what does the emulator not prove?

• What changes in the trace if R0 is initialized to 1?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 03 / INSTRUCTION AND ASSESSMENT

Field laboratory 03 — Teacher notes

Autoincrement combines a fetch and a pointer update. A fixed-size instruction encoding has limited bit positions, and expanding register selection consumes some of them. Emulated instructions can hide one or more underlying machine instructions. Similarity supports architectural comparison but not an assertion about specific design-team decisions without documentary evidence.

Before sign-off: Paired code listings, encoding sketches, a hardware-specific assembler build log, and an explicit description of architectural differences.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• Why is autoincrement in a source operand attractive for reading arrays?

• Why is a destination-mode restriction a reasonable cost of having more registers?

• What does an assembler pseudo-instruction conceal?

• What evidence would be needed to claim a direct historical design influence?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 04 / INSTRUCTION AND ASSESSMENT

Field laboratory 04 — Teacher notes

Timer_A determines edge intervals after being configured, independent of repeated CPU instructions. Display formatting cannot certify accuracy. An edge-timestamp or logic-analyzer trace can establish high/low time. Independent calibrated references or a common timebase characterization are required to attribute absolute frequency error to one clock rather than the other.

Before sign-off: Photograph or wiring sketch, instrument STATUS? transcript, program checksum, five actual frequency captures for each setting, and a table of predicted versus observed results.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• Why does the CPU’s main loop not set the edge spacing?

• Could a 1000.000 reading be wrong?

• What measurement would establish duty cycle?

• How would you distinguish target clock error from meter clock error?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 05 / INSTRUCTION AND ASSESSMENT

Field laboratory 05 — Teacher notes

For a hardware-generated signal, setup instruction count need not affect steady-state output frequency. C runtime startup is an implementation requirement, not a defect. A delay loop's timing is dependent on instruction sequences and the clock, and therefore is much more sensitive to code generation. Changes in power, clock source, pin loading, or meter configuration confound a language comparison.

Before sign-off: Build log, assembler and C source hashes, selected disassembly, hardware timer state, paired capture sheets, and an interpretation limited to the measured properties.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• Is a shorter assembly listing necessarily faster at the pin?

• Why might a compiler initialize RAM before main?

• What would change if the output were controlled by a delay loop instead of Timer_A?

• Which differences constitute an invalid comparison?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 06 / INSTRUCTION AND ASSESSMENT

Field laboratory 06 — Teacher notes

The OR assignment changes the selected bit while preserving other latch bits. Mechanical contacts bounce on short time scales. Industrial signals may require filtering, isolation, surge handling, and appropriate voltage/current interfaces. A measured voltage demonstrates an electrical state at a specific moment, not correct software behavior across all scenarios.

Before sign-off: Switch truth table with measured volts, GPIO-register snapshot or code trace, LED/pin results, and a note on wiring-dependent polarity.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• What does `P1OUT |= BIT0` preserve?

• Why can contact bounce matter even if the LED looks steady?

• Why is an internal pull-up not a general solution for industrial field wiring?

• What can a measured 3.3 V pin tell you, and what can it not tell you?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 07 / INSTRUCTION AND ASSESSMENT

Field laboratory 07 — Teacher notes

Latency runs from the external request to start of handler execution. Execution time is the handler's own work. Blocking telemetry in an ISR makes the worst-case response hard to bound. Polling may be simpler for slow or noncritical tasks. Priority can protect urgent work while starving lower-priority work if load and scheduling are not engineered carefully.

Before sign-off: Clock and deadline calculation, firmware version, GPIO timing trace, source explanation, and bounded assumptions about worst-case response.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• What is the difference between interrupt latency and interrupt execution time?

• Why should an ISR avoid expensive printing?

• When is polling preferable to interrupts?

• How might the PDP-11's priority scheme affect lower-priority industrial tasks?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 08 / INSTRUCTION AND ASSESSMENT

Field laboratory 08 — Teacher notes

Without raw values, units, conversion status, or error flags, a formatted temperature is difficult to audit. A receiver must detect and reject incomplete frames/records instead of interpreting stale fields. Full formatting libraries can consume scarce code and RAM. Missing sequence values should be explicitly reported in the observation record rather than silently interpolated.

Before sign-off: Actual serial transcript, wiring map, baud assumption and calculation, record schema, and one intentionally malformed-record handling test.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• What information is lost when a message contains only a formatted temperature?

• How should a receiver handle a truncated message?

• Why is printf size relevant on a microcontroller?

• What does the host need to do when a sequence number is missing?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 09 / INSTRUCTION AND ASSESSMENT

Field laboratory 09 — Teacher notes

Weak or poorly controlled pull-ups and wire capacitance can stretch the rising edge. Frequency and pulse width are distinct measured properties. Noise, stuck-low wiring, or a mistimed sample can impersonate presence. Blocking interrupts too long undermines system response; the critical interval must be small and explicitly justified.

Before sign-off: Waveform screenshots or time-stamped edges, measured reset and presence widths, power/pull-up measurements, and explicit device-present versus device-absent test outcomes.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• Why does internal GPIO pull-up resistance often require additional review?

• Why is 1 kHz frequency measurement not a test of a 6 µs low pulse?

• What could produce a false presence detection?

• Why should interrupts be bounded or excluded only in critical short slots?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 10 / INSTRUCTION AND ASSESSMENT

Field laboratory 10 — Teacher notes

CRC checks integrity of bytes delivered, not sensor calibration. Representation granularity is not absolute measurement uncertainty. Two's-complement interpretation is necessary for negative readings. A maximum conversion interval informs when data may legitimately be read and how the controller schedules other tasks.

Before sign-off: A nine-byte raw transaction, CRC calculation, decoded units, conversion-completion evidence, and an independent physical comparison with limitations.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• Does a valid CRC demonstrate temperature accuracy?

• What does 0.0625 °C resolution mean, and what does it not mean?

• Why must negative values be decoded as signed 16-bit?

• Why is the maximum 12-bit conversion time relevant to scheduling?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 11 / INSTRUCTION AND ASSESSMENT

Field laboratory 11 — Teacher notes

Hysteresis prevents rapid oscillation near a noisy threshold. An enumerated state is not itself a safe physical electrical level; output polarity, failure modes, and actuator behavior must be defined. Host tests isolate algorithmic transitions; they cannot validate GPIO wiring, thermal lag, sensor integrity, or interrupts. Real actuation requires protection, current limits, independent safety behavior, and process validation.

Before sign-off: Complete transition table, passing host test transcript, one injected fault, a local-output bench demonstration, and a safety limitation statement.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• What is the benefit of hysteresis over a single threshold?

• Why must the fault output action be defined independently from control state names?

• How can the host PC prove the algorithm but not prove the entire controller?

• What additional evidence is needed before using a real fan or mechanical actuator?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

INSTRUCTOR LAB 12 / INSTRUCTION AND ASSESSMENT

Field laboratory 12 — Teacher notes

Measurements depend on instruments and conditions; calculations depend on assumptions and models. Charts without raw data and setup prevent meaningful independent review. Successful 1-Wire requires electrical timing evidence plus valid protocol results, not simply a plausible temperature. Exact source revision, build command, firmware hash, instrument settings, wiring, and test date make the work recoverable.

Before sign-off: A complete CAP paper, raw data appendix, source hashes, repeatability and uncertainty explanation, feedback from an independent reviewer, and clear status labels.

Before the class begins

• Establish a controlled starting condition and a safe power/wiring configuration.

• Check the required toolchain or simulator in advance; retain the version and source files used.

• Decide what measured record will count as evidence and what observation will trigger a stop.

Ask rather than tell

• How is a measurement different from a calculated expectation?

• What is missing from a report containing only a chart?

• What is the minimum proof of a successful 1-Wire transaction?

• Which revision-control facts should survive a reboot months later?

Evidence to assess

Look for a numerical or state prediction before the test; direct observations instead of claims about intent; a separate limitation; and a clear next discriminating test.

What changes for the next lesson?

Carry forward the actual measured constraints and unresolved problems. A successful record is one that another learner could reproduce; it need not have matched the initial prediction.

APPENDIX G / REFERENCE

Working glossary and chapter locator

Addressing mode — Rule for locating operand data

Autoincrement — Update pointer after accessing pointed-to data

Bit mask — Selected bits changed or tested without altering neighbors

Control loop — Rules for observing a process and choosing actions

CRC — Check for corruption of transmitted bytes

DCO — Digitally controlled oscillator in the MSP430

Direct memory access — Peripheral data transfer without CPU handling every word

Hysteresis — Different thresholds for turning ON and OFF

Interrupt — Event that diverts the current instruction sequence

Jitter — Variation of event timing relative to its nominal schedule

Memory-mapped I/O — Hardware control registers represented in an address space

PIO — Programmable I/O state machines of RP2040

Program counter — Register identifying instruction sequencing

RSX-11M — DEC real-time multitasking operating system

RT-11 — DEC real-time operating system for the PDP-11

Scratchpad — DS18B20 register bank containing temperature and status

Timebase — Clock reference used to measure elapsed time

Traceability — Recorded chain from a claim back to raw data and standards

UNIBUS — DEC modular bus architecture

UART — Asynchronous serial transmitter/receiver

Volatile — C qualifier for objects whose value may change outside ordinary program flow

1-Wire — Single-data-wire timed bus protocol

Cross-reference map

PDP-11 hardware and industrial control: Chapters 1–3. Registers and addressing modes: Chapters 2, 4. MSP430 instruction and register effects: Chapters 4–6. Time, timer, interrupt: Chapters 5, 7. Communication and sensor protocol: Chapters 8–10. Sensor validity and safety decisions: Chapters 10–11. Evidence and publication: Chapter 12 and Appendices B–C. Teacher support: Chapters 13–14 and Appendix F.

APPENDIX H / REFERENCE TABLES

Engineering reference atlas

The following tables make an operational checklist; they do not replace the authoritative datasheet, wiring inspection, or the timing specifications of a real sensor.

REFERENCE / FIELD USE

The industrial control system — a reference contract

Signals — A transducer output has units, limits, update rate, fault behavior, physical interface, and calibration state.

Decisions — A decision maps accepted evidence to states; explicit invalid and stale input states are required.

Timing — Specify maximum acceptable response time and identify which system component enforces it.

Actuation — An output command is not evidence of physical motion; feedback may be necessary.

Safety — Emergency protection and field isolation are outside the capabilities of the introductory LED example.

Maintenance — Record revisions, board identifiers, test conditions, and a reproducible commissioning check.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

PDP-11 and MSP430 architecture — quick reference

PDP-11 R0–R5 — General working registers; R6 stack pointer, R7 program counter.

MSP430 R0 — Program counter; subject to architecture-specific instruction behavior.

MSP430 R1 — Stack pointer.

MSP430 R2 — Status register and certain special operand encodings.

MSP430 R3 — Constant-generator encodings; not a general storage register.

MSP430 R4–R15 — General working registers in the classic core.

Addressing — Always distinguish programmer-visible syntax from the encoded instruction words.

Flags — N, Z, C, and V are useful for program state; flags differ by operation and architecture.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

Bench signals and wiring — commissioning reference

MSP430 supply — Use one verified regulated 3.3 V target supply. Never connect a 5 V logic signal.

Common ground — Join MSP430 and Pico signal grounds when measuring a common-ended logic signal.

Timer output — Custom-board candidate: P1.6/TA0.1, J2 pin 7 → Pico GP3; confirm physical revision.

Pico meter — Set source input explicitly and verify STATUS; disable internal waveform generator.

UART — P1.2 TX → 3.3 V TTL adapter RX; P1.1 RX ← adapter TX; never attach RS-232 levels.

1-Wire — P1.7 candidate, 4.7 kΩ pull-up candidate; inspect the real schematic and assembly.

Debugger — Verify external-target Spy-Bi-Wire wiring and isolation jumpers with power removed.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

Frequency and error calculations — ready reference

Timer up-mode count — N=CCR0+1 clocks per up-mode period.

Toggle output — fout=fSMCLK/[2(CCR0+1)] when one toggle occurs each timer period.

Gate estimate — fhat=edge_count/elapsed_seconds; confirm edge type and actual gate measurement.

Signed difference — Delta f=fobserved−fnominal.

Relative difference — 100×Delta f/fnominal percent, when fnominal is nonzero.

Spread — Range = maximum observed − minimum observed.

Resolution versus accuracy — An extra decimal place does not imply a better timebase or traceable calibration.

Reference — An independent suitable reference is required to attribute absolute drift.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

Timing observability and error — ready reference

Event time — Timestamp where a specified physical voltage transition is observed.

Execution time — Time occupied by a specific code region, not the same as event response time.

Latency — Time from stimulus to the corresponding response; specify both boundaries.

Jitter — Variation of timing relative to its expected schedule or previous intervals.

Pulse width — Elapsed time between leading and trailing edges of one pulse.

Frequency — Rate of repetition of a periodic event; frequency alone does not prove pulse width.

Uncertainty — Include reference, resolution, clock, software gate, repeatability, and loading concerns.

Synchronization — Two instruments with separate clocks may both drift and disagree.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

1-Wire laboratory — commissioning gates

Idle — DQ must be high with a suitable pull-up and common reference.

Drive low — MCU must sink the line, not assert a high push-pull output later.

Release — MCU must stop driving so that the bus resistor can restore logic high.

Presence — An observed sensor-dependent pulse in the specified timing window.

Write slot — Separate low time for 0 and 1; test with a pulse-width-capable instrument.

Read slot — Master initiates, releases, and samples at the correct window.

Full conversion — Reset → ROM selection → conversion → wait/verify complete → reset → read scratchpad.

Data integrity — CRC of first eight scratchpad bytes compared with the ninth.

Physical measurement — Independent thermometer and thermal placement for accuracy claims.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

REFERENCE / FIELD USE

Supervisor quality assurance — reference

OFF — Output state under conditions requiring no cooling request.

ON — Output state under conditions requiring cooling request.

Hysteresis — Memory of previous valid state between upper and lower thresholds.

FAULT — Independent state for invalid, missing, or stale data; physical output action must be specified.

Boundary — Test exactly at both threshold values, not only comfortably beyond them.

No host — Unplug the observer and verify that local control continues or enters defined behavior.

Injected fault — Cause an invalid CRC or missing device and record visible fault response.

Hardware output — A logical request must be translated to a verified, protected electrical interface.

System safety — Do not present this demonstration as a qualified industrial controller.

Record your local verification

Board / instrument revision: _____

Source or datasheet and exact page: _____

Bench observation and limitation: _____

TECHNICAL VALIDATION / FINAL EDITION

What has been verified — and what has not

The final package separates a documented historical claim, an algorithm verified on a host, a target firmware candidate, and a physical experiment. These categories have different evidentiary value. A tested C function for temperature decoding does not establish correct 1-Wire waveforms or a working external sensor. A correctly printed PDF does not establish correct board wiring.

Host software verification

From the top-level archive directory, run `tests/run_host_tests.sh` in a shell with Python 3 and a C11 compiler. The script checks the CSV measurement parser, resolution-aware DS18B20 arithmetic, CRC error handling, a hysteresis state machine, and a small synthetic supervisor demonstration. The captured results are recorded under `validation/HOST_TEST_RESULTS.txt`. These tests require no MSP430 and do not involve sensor acquisition.

Target hardware verification gates

Gate 1 — Confirm board orientation, 3.3 V supply, GND, Spy-Bi-Wire reset and test connections, and isolation of the LaunchPad debug interface.

Gate 2 — Target-compile the timer assembly and C sample, preserve the compiler command/log/map file, and flash with a verified erase-main-only method.

Gate 3 — Measure five separate 1 kHz gates with live meter status and the generator disabled; preserve raw frequency, count and elapsed time.

Gate 4 — Confirm UART levels and baud, then capture protocol traces; distinguish software command from pin waveform.

Gate 5 — Validate DQ open-drain release, 1-Wire reset/presence, write slots, and read sampling with a timing-capable instrument.

Gate 6 — Demonstrate one freshly completed DS18B20 conversion, full nine-byte scratchpad, CRC, sensor identity, and independent temperature comparison.

Gate 7 — Operate the educational LED supervisor under valid, stale, missing, and erroneous data; document reset and safe-state behavior.

Hardware experiment record

Date/time and operator: _____

Board revision and firmware hash: _____

Compiler version and command: _____

Measurement firmware and live settings: _____

Raw data file and checksum: _____

Observed failure or uncertainty: _____

A completed log is the prerequisite for changing a target firmware status from CANDIDATE to BENCH-VERIFIED. This edition makes no such upgrade without the required observations.